



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра № 231 Информационных процессов и систем

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой  Н.П. Стариков
«01» июня 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по направлению подготовки магистров 09.04.04 Программная инженерия

На тему: Сравнительный анализ и разработка технологических решений для построения
конвейеров данных в распределенных информационных системах

Обучающийся


подпись

Жизневский Никита Вячеславович
Фамилия, имя, отчество

шифр 24И1560
группа ИКМО-02-24

Руководитель работы


подпись

доцент

Труфанов В.Н.

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий

Кафедра № 231 Информационных процессов и систем

СОГЛАСОВАНО

Заведующий
кафедрой


подпись

Стариков Павел Павлович

«27» февраля 2026 г.

УТВЕРЖДАЮ

И.о. директора
института


подпись

Уланский Александр Борисович

«27» февраля 2026 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы магистра
(магистерской диссертации)

Обучающийся

Жизневский Никита Вячеславович

Фамилия Имя Отчество

Шифр

24И1560

Направление
подготовки

09.04.04

индекс направления

Программная инженерия

наименование направления

Группа

ИКМО-02-24

1. Тема выпускной квалификационной работы: Сравнительный анализ и разработка технологических решений для построения конвейеров данных в распределенных информационных системах

2. Цель и задачи выпускной квалификационной работы

Цель работы: разработка, экспериментальная оценка и сравнение подходов к построению конвейеров данных с последующим формированием модели рекомендаций по их применению в распределённых информационных системах

Задачи работы: провести анализ предметной области и подходов к передаче данных; спроектировать и реализовать решения на основе Transactional Outbox, Spring Batch и Debezium CDC; разработать методику экспериментальной оценки; выполнить нагрузочное тестирование реализованных решений; проанализировать метрики производительности; составить модель рекомендаций по выбору подхода с учётом характеристик системы и ограничений инфраструктуры; подготовить презентационный материал

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Исследовательский раздел		
1.1	Анализ предметной области		
1.1.1	Особенности передачи изменений данных		
1.1.2	Требования к доставке данных		
1.2	Характеристика объекта и субъекта исследования		
1.2.1	Характеристика объекта исследования		
1.2.2	Характеристика субъекта исследования		
1.2.3	Развитие требований к синхронизации данных и ограничения применяемых решений		21.03.2026
1.3	Анализ подходов, использованных на проекте		
1.3.1	Транзакционный подход на основе паттерна Outbox		
1.3.2	Пакетная обработка на основе Spring Batch		
1.3.3	Потоковая CDC-репликация на основе Debezium		
1.4	Обоснование необходимости сравнительного анализа		
1.5	Выводы по разделу		
2	Проектный раздел		
2.1	Постановка задачи проектирования решений для передачи изменений данных		
2.1.1	Цели и ограничения		
2.1.2	Упрощения относительно корпоративного проекта		
2.1.3	Критерии сравнения подходов		
2.2	Проектирование транзакционного подхода на основе паттерна Outbox		18.04.2026
2.3	Проектирование пакетной обработки на основе Spring Batch		
2.4	Проектирование потоковой CDC-репликации на основе Debezium		
2.5	Выводы по разделу		
3	Технологический раздел		
3.1	Реализация транзакционного подхода на основе паттерна Outbox		
3.2	Реализация пакетной обработки на основе Spring Batch		
3.3	Реализация потоковой CDC-репликации на основе Debezium		
3.4	Методика проведения эксперимента		
3.4.1	Тестовые сценарии		
3.4.2	Настройка тестовой среды		
3.5	Результаты эксперимента и анализ метрик производительности		16.05.2026
3.5.1	Результаты по сценарию нагрузки для транзакционного решения на основе паттерна Outbox		
3.5.2	Результаты по сценарию нагрузки для пакетной обработки на основе Spring Batch		
3.5.3	Результаты по сценарию нагрузки для CDC репликации на основе Debezium		
3.6	Составление модели рекомендаций применения подходов		
3.7	Выводы по разделу		

4	Введение, заключение, список источников, приложения		18.05.2026
5	Презентационный материал, аннотация		27.05.2026
6	Нормоконтроль		28.05.2026

4. Перечень разрабатываемых документов и графических материалов: электронная версия магистерской диссертации, презентационный материал с основными результатами выпускной квалификационной работы магистра

5. Руководитель и консультант выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	Доцент	Труфанов Владимир Николаевич	
Задание выдал Руководитель ВКР: _____	 подпись	Задание принял к исполнению Обучающийся: _____	 подпись
«27» февраля 2026 г.		«27» февраля 2026 г.	

Руководитель ВКР: доцент В.Н. Труфанов

Жизневский Н.В., Выпускная квалификационная работа по образовательной программе «Архитектура информационных систем» направления подготовки «Программная инженерия» на тему «технологических решений для построения конвейеров данных в распределенных»: М. 2026 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра информационных процессов и систем (ИПиС) – 93 стр., 27 рис, 14 лист., 2 табл., 37 источн..

Ключевые слова: передача изменений данных; распределённая информационная система; Transactional Outbox; Spring Batch; Debezium CDC; Apache Kafka; PostgreSQL; нагрузочное тестирование; модель рекомендаций.

Целью работы является разработка, экспериментальная оценка и сравнение подходов к построению конвейеров данных с последующим формированием модели рекомендаций по их применению в распределённых информационных системах.

N.V. Zhiznevsky, Graduation Thesis for the Educational Program "Information Systems Architecture," Major in Software Engineering, on the Topic of "Technological Solutions for Building Data Pipelines in Distributed Systems": Moscow, 2026, MIREA – Russian Technological University (RTU MIREA), Institute of Information Technology (IIT), Department of Information Processes and Systems (IPiS) – 93 pp., 27 figs., 14 sheets., 2 tables, 37 sources.

Keywords: data change transfer; distributed information system; Transactional Outbox; Spring Batch; Debezium CDC; Apache Kafka; PostgreSQL; load testing; recommendation model.

The aim of this work is to develop, experimentally evaluate, and compare approaches to building data pipelines, followed by the formation of a recommendation model for their use in distributed information systems.

РТУ МИРЭА: 119454, Москва, пр-т Вернадского, д. 78
кафедра информационных процессов и систем (ИПиС)

Тираж: 1 экз. (на правах рукописи)

Файл: «», исполнитель Жизневский Н.В.

© Н.В. Жизневский

АННОТАЦИЯ

Тема выпускной квалификационной работы: «Сравнительный анализ и разработка технологических решений для построения конвейеров данных в распределённых информационных системах».

Выпускная квалификационная работа включает введение, три раздела, заключение, список использованных источников и приложения.

В Исследовательском разделе рассмотрены особенности передачи изменений данных в распределённых информационных системах, требования к доставке данных, характеристика объекта и субъекта исследования, а также подходы Transactional Outbox, Spring Batch и Debezium CDC. Обоснована необходимость их сравнительного анализа в единых условиях.

В Проектном разделе сформулирована задача проектирования решений, определены цели, ограничения и критерии сравнения. Спроектированы три технологических решения для передачи изменений данных. Для описания их структуры и взаимодействия компонентов разработаны архитектурные схемы, sequence-диаграммы и ERD-модели.

В Технологическом разделе описаны реализация решений, настройка тестовой среды, методика проведения эксперимента и сценарии нагрузочного тестирования. Проведён анализ метрик производительности, на основе которого сформирована модель рекомендаций по выбору подхода с учётом характеристик системы и ограничений инфраструктуры.

Выпускная квалификационная работа состоит из 92 страницы и включает 27 рисунков, 2 таблицы, 14 листингов, список источников на 37 позиций и 5 приложений.

Ключевые слова: Transactional Outbox; Spring Batch; Debezium CDC; Apache Kafka; PostgreSQL; нагрузочное тестирование; модель рекомендаций.

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....	9
ВВЕДЕНИЕ.....	10
ЛИТЕРАТУРНЫЙ ОБЗОР	13
1. ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ	14
1.1 Анализ предметной области	14
1.1.1 Особенности передачи изменений данных	14
1.1.2 Требования к доставке данных	15
1.2 Характеристика объекта и субъекта исследования	17
1.2.1 Характеристика объекта исследования.....	17
1.2.2 Характеристика субъекта исследования.....	18
1.2.3 Развитие требований к синхронизации данных и ограничения применяемых решений	19
1.3 Анализ подходов, использованных на проекте.....	21
1.3.1 Транзакционный подход на основе паттерна Outbox.....	21
1.3.2 Пакетная обработка на основе Spring Batch.....	23
1.3.3 Потокковая CDC-репликация на основе Debezium	24
1.4 Обоснование необходимости сравнительного анализа.....	25
1.5 Выводы по разделу.....	27
2. ПРОЕКТНЫЙ РАЗДЕЛ.....	28
2.1 Постановка задачи проектирования решений для передачи изменений данных	28
2.1.1 Цели и ограничения	28
2.1.2 Упрощения относительно корпоративного проекта	29
2.1.3 Критерии сравнения подходов.....	30
2.2 Проектирование транзакционного подхода на основе паттерна Outbox	32
2.3 Проектирование пакетной обработки на основе Spring Batch	36
2.4 Проектирование потокковой CDC-репликации на основе Debezium.....	38
2.5 Выводы по разделу.....	40
3. ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	41

3.1 Реализация транзакционного подхода на основе паттерна Outbox	41
3.2 Реализация пакетной обработки на основе Spring Batch	46
3.3 Реализация потоковой CDC-репликации на основе Debezium	49
3.4 Методика проведения эксперимента.....	52
3.4.1 Тестовые сценарии.....	53
3.4.2 Настройка тестовой среды	54
3.5 Результаты эксперимента и анализ метрик производительности.....	56
3.5.1 Результаты по сценарию нагрузки для транзакционного решения на основе паттерна Outbox.....	56
3.5.2 Результаты по сценарию нагрузки для пакетной обработки на основе Spring Batch.....	59
3.5.3 Результаты по сценарию нагрузки для CDC репликации на основе Debezium	63
3.6 Составление модели рекомендаций применения подходов	67
3.7 Выводы по разделу.....	68
ЗАКЛЮЧЕНИЕ	69
CONCLUSION	70
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОНИЧКОВ	71
ПРИЛОЖЕНИЯ.....	75

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

Термин	Значение
БД	база данных
ИТ	информационные технологии
СУБД	система управления базами данных
C4-диаграмма	Context, Containers, Components, Code, модель описания архитектуры программных систем
CDC	Change Data Capture, технология фиксации и передачи изменений данных
CPU	Central Processing Unit, центральный процессор
CRM	Customer Relationship Management, система управления взаимоотношениями с клиентами
DWH	Data Warehouse, хранилище данных
ER-диаграмма	Entity–Relationship diagram, диаграмма «сущность–связь»
I/O	Input/Output, операции ввода-вывода
JDBC	Java Database Connectivity, интерфейс взаимодействия Java-приложений с базами данных
JVM	Java Virtual Machine, виртуальная машина Java

ВВЕДЕНИЕ

В корпоративных информационных системах всё чаще возникает необходимость передавать изменения данных из операционного контура в аналитический. При этом важно учитывать не только сам факт доставки, но и задержку передачи, влияние выбранного механизма на бизнес-сервис и базу данных, устойчивость системы под нагрузкой, а также сложность сопровождения решения.

Актуальность работы связана с тем, что в реальных проектах один подход к передаче изменений данных не всегда остаётся эффективным на всех этапах развития системы. По мере роста нагрузки, изменения требований к актуальности данных и расширения интеграционного контура возникает необходимость пересматривать архитектурные решения. В рамках рассматриваемого проекта применялись три подхода: Transactional Outbox [1-4], пакетная обработка на основе Spring Batch [5-7] и CDC-репликация на основе Debezium [8-9]. Каждый из них имеет свои преимущества и ограничения, поэтому требуется их сравнение в единых условиях.

Целью выпускной квалификационной работы является сравнение трёх подходов к передаче изменений данных на основе их реализации, нагрузочного тестирования и анализа влияния на бизнес-сервис, базу данных и интеграционную инфраструктуру.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и подходов к передаче данных;
- спроектировать и реализовать решения на основе Transactional Outbox, Spring Batch и Debezium CDC;
- разработать методику экспериментальной оценки;
- выполнить нагрузочное тестирование реализованных решений;
- проанализировать метрики производительности;

– составить модель рекомендаций по выбору подхода с учётом характеристик системы и ограничений инфраструктуры.

Объектом исследования является процесс передачи изменений данных из операционного контура во внешний аналитический контур в условиях распределённой информационной системы.

Субъектом исследования выступает распределённая информационная система, в рамках которой реализуются, тестируются и сравниваются разные механизмы синхронизации данных.

Научная новизна работы заключается в формировании сравнительной модели выбора подхода к передаче изменений данных на основе экспериментальной оценки трёх архитектурных решений в единых условиях. В отличие от общего описания отдельных технологий, в работе сопоставляются Transactional Outbox, Spring Batch и Debezium CDC по влиянию на бизнес-сервис, базу данных, Kafka. Это позволяет определить не только производительные характеристики каждого подхода, но и условия, при которых его применение является наиболее обоснованным.

In corporate information systems, there is an increasing need to transfer data changes from the operational to the analytical environment. This requires considering not only the actual delivery process, but also the transfer latency, the impact of the chosen mechanism on the business service and database, system stability under load, and the complexity of solution maintenance.

The relevance of this work stems from the fact that in real-world projects, a single approach to transferring data changes is not always effective at all stages of system development. As the workload increases, data relevance requirements change, and the integration environment expands, it becomes necessary to reconsider architectural solutions. Three approaches were used in this project: Transactional Outbox [1-4], batch processing based on Spring Batch [5-7], and CDC replication based on Debezium [8-9]. Each has its own advantages and limitations, so a comparison under consistent conditions is required.

The goal of this final thesis is to compare three approaches to transferring data changes based on their implementation, load testing, and impact analysis on the business service, database, and integration infrastructure. To achieve this goal, the following tasks must be completed:

- Conduct an analysis of the subject area and data transfer approaches;
- Design and implement solutions based on Transactional Outbox, Spring Batch, and Debezium CDC;
- Develop an experimental evaluation methodology;
- Perform load testing of the implemented solutions;
- Analyze performance metrics;
- Develop a model for recommending an approach based on system characteristics and infrastructure constraints.

The object of this study is the process of transferring data changes from the operational environment to the external analytical environment within a distributed information system.

The subject of this study is a distributed information system within which various data synchronization mechanisms are implemented, tested, and compared.

The scientific novelty of this work lies in the development of a comparative model for selecting an approach to transferring data changes based on an experimental evaluation of three architectural solutions under similar conditions. Rather than a general description of individual technologies, this work compares Transactional Outbox, Spring Batch, and Debezium CDC based on their impact on the business service, database, and Kafka. This allows us to determine not only the performance characteristics of each approach but also the conditions under which its use is most justified.

ЛИТЕРАТУРНЫЙ ОБЗОР

Основу теоретической части составили материалы С. Richardson, AWS и G. M. Hafiz, посвящённые паттерну Transactional Outbox. В данных источниках описывается подход, при котором изменение бизнес-данных и запись события на отправку выполняются согласованно в рамках одной транзакции. Эти материалы использовались при рассмотрении транзакционного решения и его роли в обеспечении надёжной передачи изменений данных.

Для изучения пакетной обработки использовалась официальная документация Spring Batch. В ней раскрываются принципы построения пакетных заданий, организация обработки данных по шагам, хранение служебного состояния выполнения и возможности масштабирования. Эти источники стали основой для описания решения, при котором передача изменений выполняется по расписанию.

CDC-подход рассматривался на основе материалов IBM, Red Hat, Debezium и Confluent. В них описывается технология фиксации изменений данных, чтение изменений из журнала транзакций базы данных и передача событий через Kafka Connect. Эти источники использовались при анализе потоковой репликации изменений данных.

Для проектирования решений применялись материалы по модели C4, sequence-диаграммам и ER-диаграммам. Они позволили формализовать архитектуру решений, показать состав компонентов и описать взаимодействие между сервисами, базами данных и интеграционной инфраструктурой.

Технологическая и экспериментальная части работы опирались на документацию Docker Compose, k6, Prometheus, Grafana и Spring Boot Actuator. Эти источники использовались при описании тестовой среды, сценариев нагрузочного тестирования, сбора метрик и анализа производительности реализованных решений.

1. ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

1.1 Анализ предметной области

Перед рассмотрением практической части нужно определить, что именно представляет собой передача изменений данных в распределённой системе и какие требования предъявляются к таким механизмам.

1.1.1 Особенности передачи изменений данных

Задача передачи изменений возникает довольно часто. Особенно когда одни и те же данные нужны сразу в нескольких частях системы. Например, операционное приложение работает с текущими операциями, а отдельно есть ещё аналитика, отчётность или внешние сервисы. Чтобы всё это не расходилось, нужно как-то передавать изменения из источника в другие контуры.

На практике редко выгружают все данные целиком при каждом обновлении - это дорого и медленно. Вместо этого передают только то, что изменилось за определённое время или в рамках конкретных операций. То есть внешний контур получает только новые, изменённые или удалённые записи.

Самый типичный пример - аналитика. Берём изменения по обращениям клиентов, статусам, действиям сотрудников и передаём их в хранилище данных. А оттуда уже строим отчётность, считаем показатели, делаем витрины данных. Витрина - это просто набор данных по конкретной теме: витрина по обращениям, по инцидентам, по активности пользователей. То есть аналитик получает уже не сырые таблицы, а готовую, структурированную информацию.

Второй распространённый случай не аналитика, а интеграция между системами. Если данные поменялись в одной системе - это должно привести к обновлению информации в другом сервисе, отправке уведомления, запуску какого-то процесса. Например, изменился статус заявки - и сразу надо об этом сказать сервису уведомлений или системе мониторинга. Здесь передача изменений уже выступает как часть межсистемного взаимодействия.

Таким образом, передача изменений - это обычная задача для распределённых систем. Она возникает на стыке операционной, аналитической и интеграционной частей. И используется везде, где нужно регулярно и управляемо отдавать наружу то, что поменялось в источнике.

1.1.2 Требования к доставке данных

Но сам по себе факт, что изменение передалось, ещё не означает, что синхронизация работает хорошо. На практике важны и другие вещи: как быстро изменение доходит до нужной системы, какую нагрузку это создаёт, выдерживает ли механизм рост потока данных и насколько сложно его потом сопровождать [10-11]. Поэтому, когда говорят о передаче изменений, обычно оценивают не просто возможность доставки, а ещё и её качество (Таблица 1.1).

Таблица 1.1 – Основные требования к доставке данных

Требование	Содержание требования	Практическое значение
Задержка доставки	Интервал между моментом фиксации изменения в источнике и его поступлением в целевую систему	Позволяет оценить, подходит ли решение для пакетных или близких к реальному времени сценариев
Влияние на источник данных	Степень дополнительной нагрузки на прикладной сервис и связанную с ним базу данных	Показывает, допустимо ли применение подхода в продуктивной среде без ухудшения работы основной системы
Нагрузка на инфраструктурные компоненты	Интенсивность нагрузки на брокер сообщений, коннекторы, фоновые процессы и другие элементы интеграционного контура	Позволяет оценить устойчивость решения при росте потока данных
Масштабируемость	Способность решения сохранять работоспособность при увеличении числа операций и объёма передаваемых данных	Важна для развивающихся и высоконагруженных систем
Корректность обработки	Допустимый режим доставки сообщений, порядок обработки изменений, устойчивость к повторной доставке	Обеспечивает согласованность данных между источником и потребителем

Задержка доставки - это время между тем, когда изменение зафиксировалось в источнике, и тем, когда оно появилось в целевой системе. Для некоторых задач такая задержка не критична. Например, если данные раз в час грузятся в аналитическое хранилище, небольшая задержка нормальна. Но в других случаях даже маленький лаг может быть проблемой - ценность данных от этого падает.

Влияние на источник данных. Здесь смотрят, насколько механизм синхронизации нагружает прикладной сервис и базу данных. Если из-за передачи изменений начинает тормозить основная транзакционная работа - такое решение вряд ли подойдёт для реальной эксплуатации.

Нагрузка на инфраструктуру - это про брокеры сообщений, коннекторы, фоновые процессы и всё, что крутится вокруг интеграции. Бывает, что сам источник загружен не сильно, а узкое место возникает именно здесь. С этим же связана масштабируемость: решение должно сохранять работоспособность, когда число операций и объём данных растут.

Корректность обработки. Тут смотрят, как система ведёт себя при доставке сообщений. Обычно выделяют три модели: at-most-once, at-least-once и exactly-once [12]. Первая - сообщение обработается максимум один раз, но может потеряться. Вторая - доставка будет хотя бы один раз, но могут появиться дубликаты, так что нужна идемпотентность. Третья - однократная обработка без потерь и дублирования, но это сложнее в реализации.

Кроме того, важна сложность сопровождения. Даже если решение отличное с технической точки зрения, оно может оказаться неудачным, если его трудно внедрять, настраивать и потом поддерживать.

Все перечисленные требования взаимосвязаны. Попытка уменьшить задержку обычно ведёт к усложнению инфраструктуры. Пакетная передача, наоборот, снижает постоянную нагрузку на источник, зато задержка растёт и могут возникать пиковые нагрузки. А если усилить гарантии корректности, то реализация и сопровождение тоже усложняются.

1.2 Характеристика объекта и субъекта исследования

После рассмотрения предметной области необходимо определить объект и субъект исследования. Это позволяет перейти от общего описания передачи изменений данных в распределённых системах к анализу конкретной практической ситуации, в рамках которой данная задача решается в реальном корпоративном проекте.

1.2.1 Характеристика объекта исследования

Объектом исследования является процесс передачи изменений данных из операционного контура во внешний аналитический контур в условиях распределённой информационной системы. Этот процесс включает фиксацию изменений в источнике, их выделение из общего потока операций, передачу через интеграционный механизм и последующее поступление в целевую систему для дальнейшего хранения, обработки и анализа.

Практическая значимость данного процесса связана с тем, что в современных информационных системах операционный и аналитический контуры, как правило, разделены. Операционная система обеспечивает выполнение прикладных бизнес-процессов, обработку действий пользователей и хранение актуального состояния данных. Внешний аналитический контур используется для построения отчётности, подготовки витрин данных, расчёта агрегированных показателей и последующего анализа накопленной информации [13-14]. По этой причине изменения, возникающие в операционном источнике, должны передаваться во внешний контур достаточно полно, регулярно и предсказуемо.

Процесс передачи изменений данных нельзя свести только к технической пересылке записей из одной базы данных в другую. На практике он затрагивает несколько уровней системы одновременно. На прикладном уровне он связан с тем, как и в какой момент изменение считается значимым для внешнего контура. На уровне хранения данных он определяется способом фиксации изменений в

базе данных. На интеграционном уровне он зависит от механизма извлечения, преобразования и доставки данных в целевую систему. Наконец, на эксплуатационном уровне он определяется тем, какую нагрузку создаёт выбранный механизм и насколько устойчиво он работает при росте объёма данных.

С точки зрения исследования данный объект представляет интерес потому, что один и тот же процесс передачи изменений может быть реализован различными способами. В зависимости от выбранного архитектурного подхода будет различаться момент фиксации события, способ его извлечения, режим передачи данных и состав инфраструктурных компонентов, участвующих в синхронизации. Это, в свою очередь, влияет на задержку доставки, нагрузку на источник, устойчивость интеграционного контура и сложность сопровождения решения.

1.2.2 Характеристика субъекта исследования

Субъектом исследования выступает корпоративный проект ЛКС («Личный кабинет сотрудника») - CRM-система, предназначенная для сопровождения клиентов сегмента малого и среднего бизнеса. Система используется для поддержки операционной деятельности сотрудников, обработки обращений, взаимодействия с внутренними банковскими сервисами и внешними платформами, а также для передачи данных во внешний аналитический контур.

Основными пользователями системы являются сотрудники банка, работающие с клиентскими обращениями, документами, инцидентами и сопутствующими операционными процессами. В зависимости от функциональной роли пользователи обращаются к системе для регистрации и обработки обращений, получения клиентской информации, сопровождения внутренних процедур и взаимодействия со смежными сервисами. Таким образом, ЛКС выступает рабочим инструментом для выполнения повседневных прикладных задач и одновременно точкой интеграции с другими информационными системами организации.

Архитектура проекта построена на основе микросервисного подхода. В состав системы входит около 20 сервисов, реализованных на Java и взаимодействующих между собой в рамках общего интеграционного контура. Такая архитектура обеспечивает разделение ответственности между компонентами, однако одновременно повышает требования к организации обмена данными между сервисами и внешними системами.

Особое значение в рамках проекта имеет передача данных в DWH. Внешний аналитический контур используется для формирования отчётности, построения витрин данных и анализа накопленной информации. Из-за этого механизм синхронизации должен обеспечивать не только саму передачу изменений, но и достаточную полноту, согласованность и предсказуемость поступления данных. Недостатки такого механизма отражаются не только на техническом состоянии интеграционного контура, но и на качестве аналитической информации, используемой во внешних системах.

С практической точки зрения проект представляет интерес как реальная распределённая система, в которой задача передачи изменений данных не является второстепенной. Она напрямую связана с обеспечением работы аналитического контура и зависит как от прикладных требований, так и от инфраструктурных ограничений. Именно это делает ЛКС подходящим субъектом исследования для рассмотрения различных подходов к синхронизации данных и их дальнейшего сопоставления.

1.2.3 Развитие требований к синхронизации данных и ограничения применяемых решений

В ходе развития системы требования к механизму синхронизации данных постепенно менялись. На начальном этапе ключевое значение имела гарантированная передача изменений в DWH при сохранении согласованности между операционной базой данных и внешним аналитическим контуром. Объёмы данных на этом этапе оставались умеренными, поэтому основное

внимание уделялось надёжности решения и контролю корректности передачи (Рисунок 1.1).

По мере развития проекта изменилась сама эксплуатационная среда. Увеличилось количество сервисов, расширилась бизнес-функциональность, возросло число операций и интеграционных взаимодействий. В таких условиях стало важным не только обеспечить передачу изменений, но и сократить влияние механизма синхронизации на прикладные сервисы, базу данных и смежные инфраструктурные компоненты. Иначе интеграционный контур начинал оказывать заметное влияние на производственную систему, для которой он изначально являлся вспомогательным.

На следующем этапе всё большее значение приобрела задержка доставки данных. Для аналитического контура стало важно не просто получать изменения, а получать их в более короткие сроки, чтобы уменьшить временной разрыв между состоянием операционной системы и состоянием данных во внешней среде. Одновременно возникла необходимость избегать неравномерной передачи, при которой накопленные изменения поступают крупными пакетами и создают локальные пики нагрузки на инфраструктуру.



Рисунок 1.1 - Эволюция требований к синхронизации данных на проекте [разработано автором]

Таким образом, по мере развития системы менялись не только используемые технические решения, но и сами приоритеты требований. Если на раннем этапе на первый план выходили согласованность и надёжность, то затем всё большее значение стали иметь задержка доставки, масштабируемость, устойчивость под нагрузкой и допустимый уровень эксплуатационной

сложности. Это означало, что механизм синхронизации должен оцениваться уже не по одному критерию, а по совокупности характеристик.

Практический опыт проекта показал, что ни один из применявшихся механизмов не обеспечивал универсального решения задачи передачи изменений данных. Каждый подход позволял устранить часть ограничений предыдущего этапа, однако одновременно формировал новые компромиссы. Одни решения лучше обеспечивали согласованность фиксации изменений и их последующей передачи, но сильнее нагружали источник данных. Другие уменьшали влияние на прикладную систему, но увеличивали задержку доставки. Третьи позволяли приблизиться к режиму реального времени, но требовали более сложной инфраструктуры и более высокого уровня сопровождения.

В результате основная проблемная ситуация заключалась не в отсутствии работоспособного механизма передачи данных как такового, а в необходимости выбрать такой подход, который обеспечивал бы приемлемый баланс между несколькими требованиями сразу.

1.3 Анализ подходов, использованных на проекте

1.3.1 Транзакционный подход на основе паттерна Outbox

Транзакционный подход на основе паттерна Outbox предполагает фиксацию информации об изменении одновременно с сохранением основных бизнес-данных. В рамках одной транзакции в базе данных выполняются две операции: изменение или сохранение бизнес-сущности и запись соответствующего события в специальную outbox-таблицу. Таким образом, данные об изменении и информация, необходимая для последующей передачи, фиксируются согласованно в одном транзакционном контексте [7].

После завершения транзакции отдельный компонент системы выполняет чтение записей из outbox-таблицы. Вычитанные сообщения преобразуются в необходимый формат и передаются в брокер сообщений либо иной внешний

контур (Рисунок 1.2). Далее обработанные записи могут помечаться как отправленные или удаляться в зависимости от выбранной реализации.

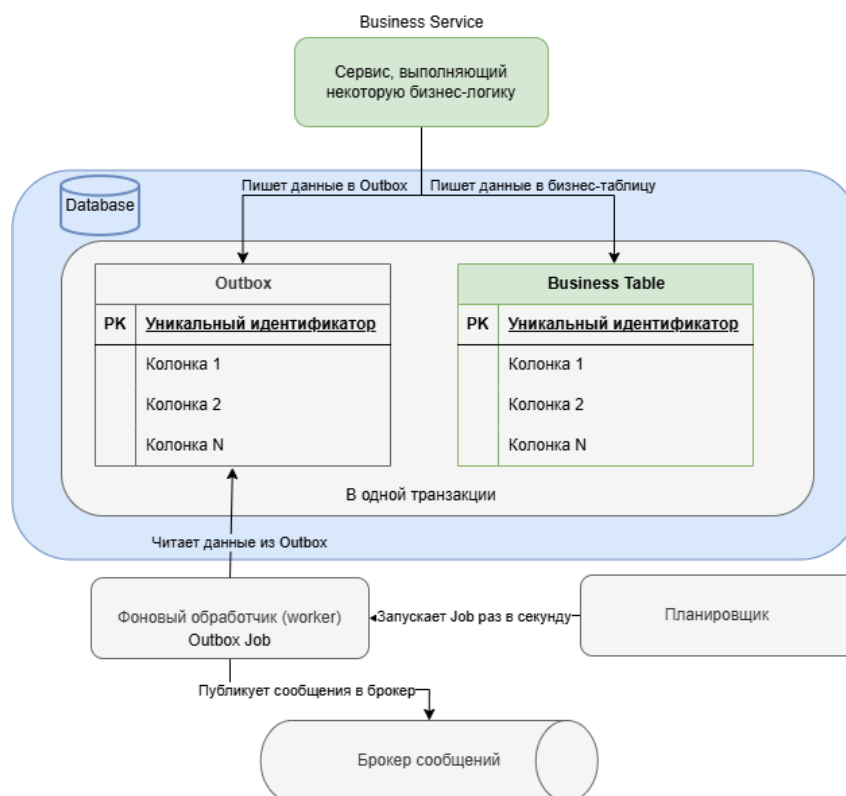


Рисунок 1.2 – Схема работы Transactional Outbox подхода [разработано автором]

Ключевым достоинством данного подхода является согласованная фиксация бизнес-изменения и события на отправку без необходимости использования распределённых транзакций. Вместе с тем применение Outbox создаёт дополнительную нагрузку на прикладную базу данных, связанную с записью, хранением и последующей обработкой outbox-событий.

Следует отметить, что помимо классической реализации с отдельной outbox-таблицей существует и альтернативный вариант применения данного подхода. В нём событие на отправку не выносится в отдельную таблицу, а фиксируется непосредственно рядом с бизнесовой записью, например с помощью специального признака, статуса или флага обработки. В этом случае основная таблица содержит не только бизнес-данные, но и служебную информацию о необходимости последующей передачи изменения. Отдельный планировщик или фоновая задача периодически просматривает такие записи,

отбирает ещё не обработанные изменения, отправляет их во внешний контур и после успешной отправки изменяет флаг или статус обработки.

В рамках данной работы рассматривается реализация паттерна Transactional Outbox с выделенной outbox-таблицей. Такой вариант был выбран, поскольку именно он использовался в исследуемом проекте. В этой реализации бизнес-данные и события на отправку сохраняются в рамках одной транзакции, но хранятся отдельно: основная сущность размещается в бизнес-таблице, а сообщение для последующей доставки - в outbox-таблице. Это позволяет отделить интеграционный контур от структуры основной бизнес-модели и более явно контролировать процесс публикации событий.

1.3.2 Пакетная обработка на основе Spring Batch

Пакетная обработка на основе Spring Batch предполагает передачу изменений не непрерывно, а через определённые интервалы времени в составе заранее сформированных наборов данных. В рамках данного подхода изменения, возникшие за определённый период, накапливаются в источнике данных, после чего запускается отдельный процесс пакетной обработки.

Логика работы пакетного подхода заключается в последовательном выполнении этапов чтения данных, их обработки и записи результата. При необходимости общий объём данных разбивается на отдельные части, что позволяет организовать обработку крупных массивов информации и лучше контролировать использование ресурсов. Запуск процесса может выполняться по расписанию, например в заранее определённое время суток (Рисунок 1.3).

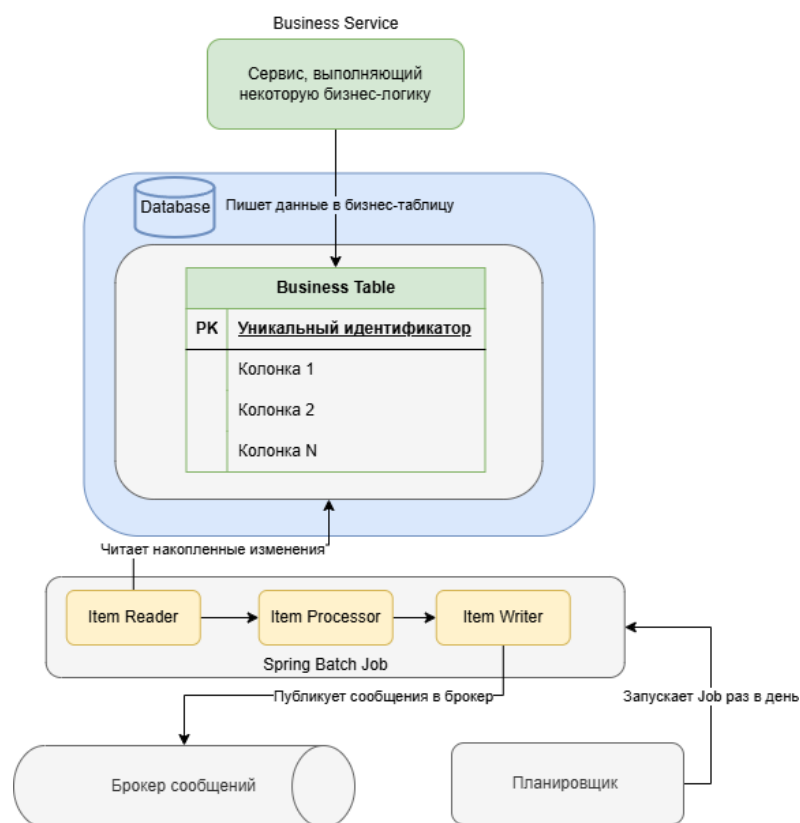


Рисунок 1.3 – Схема работы Spring Batch подхода [разработано автором]

Преимуществами пакетного подхода являются управляемость момента запуска, возможность переноса обработки на периоды минимальной пользовательской активности и удобство контроля состояния выполнения заданий. Вместе с тем пакетная обработка сопровождается ростом задержки доставки и риском формирования кратковременных пиков нагрузки в момент запуска процесса.

1.3.3 Поточковая CDC-репликация на основе Debezium

Поточковая CDC-репликация на основе Debezium основана на извлечении изменений непосредственно из журнала транзакций базы данных. В данном случае источником событий выступает не прикладная логика сервиса, а инфраструктурный механизм фиксации изменений на уровне СУБД.

Логика работы подхода заключается в том, что изменения, происходящие в таблицах базы данных, фиксируются системой управления базами данных в журнале транзакций. Инструмент Debezium подключается к этому журналу,

считывает сведения о вставке, обновлении и удалении записей, после чего формирует поток событий на основе обнаруженных изменений. Сформированные события передаются во внешний брокер сообщений, откуда могут быть получены и обработаны целевыми системами (Рисунок 1.4).

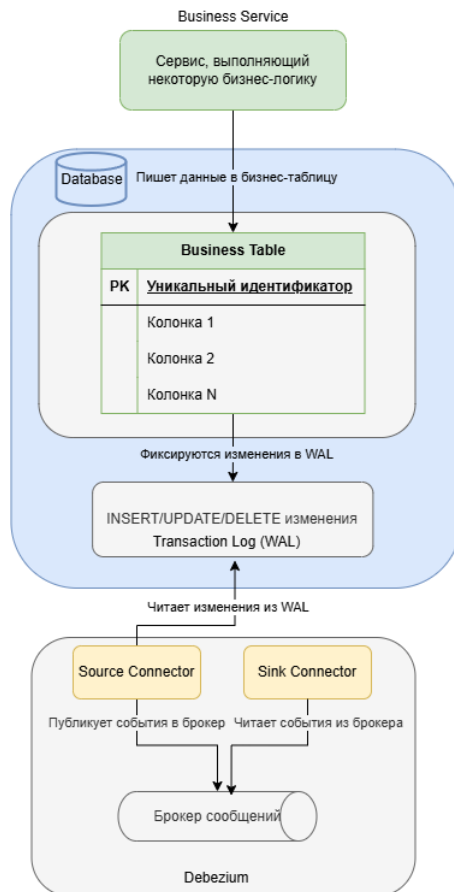


Рисунок 1.4 – Схема работы CDC подхода [разработано автором]

Принципиальное отличие CDC-подхода состоит в том, что извлечение изменений осуществляется не на уровне прикладной логики сервиса, а на уровне инфраструктуры базы данных. Это позволяет уменьшить влияние механизма передачи на приложение, однако переносит существенную часть сложности в область настройки и сопровождения инфраструктурных компонентов.

1.4 Обоснование необходимости сравнительного анализа

Рассмотренные подходы направлены на решение одной и той же задачи - передачу изменений данных из операционного контура во внешний аналитический. Однако различие между ними состоит не только в используемых

технологиях, но и в самом способе организации процесса синхронизации (Рисунок 1.5). В одном случае изменение фиксируется на уровне прикладной транзакции, в другом - передаётся в составе периодически формируемого пакета, в третьем - извлекается из журнала транзакций СУБД. Эти различия затрагивают момент фиксации изменения, способ его извлечения, режим доставки и состав инфраструктурных компонентов, участвующих в обработке.

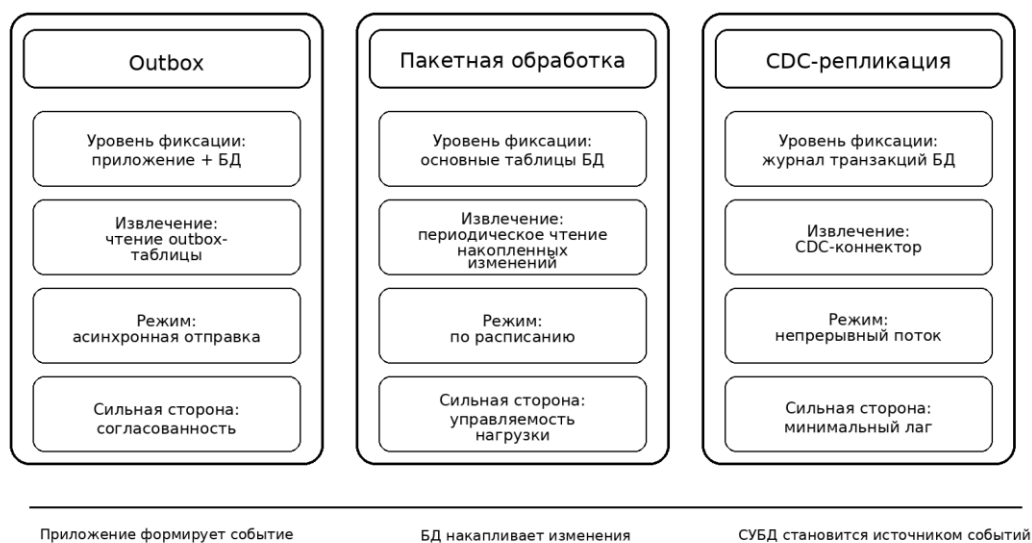


Рисунок 1.5 - Сопоставление подходов к передаче изменений данных [разработано автором]

По этой причине простого описания принципов работы подходов недостаточно. Само по себе описание позволяет понять логику решения, но не даёт ответа на более важный практический вопрос: каким будет поведение этого решения в условиях реальной эксплуатации. Из описания нельзя напрямую определить, насколько сильно подход повлияет на сервис-источник, какую нагрузку создаст на базу данных и инфраструктурный контур, как изменится его эффективность при росте объёма передаваемых данных и насколько сложным окажется его сопровождение.

Следовательно, выбор механизма передачи изменений данных не может основываться только на популярности технологии, общих архитектурных соображениях или отдельных теоретических преимуществах. Для обоснованного выбора необходимо понять, как именно рассматриваемые

подходы проявляют себя по совокупности значимых характеристик: задержке доставки, влиянию на сервис-источник, нагрузке на базу данных, устойчивости инфраструктурного контура и сложности сопровождения. Только такое сопоставление позволяет перейти от формального описания решений к оценке их практической применимости. Для этого необходимо использовать общую модель источника данных, единый целевой контур, одинаковую логику передачи изменений и единый набор критериев оценки. Только при таком подходе различия в результатах будут определяться особенностями самих архитектурных решений, а не различиями предметной логики, структуры данных или внешней среды эксперимента.

Таким образом, сравнительный анализ в рамках настоящего исследования необходим для того, чтобы определить не просто различие подходов по способу реализации, а различие их эксплуатационных и производительных характеристик в единых условиях. Это позволяет сформировать обоснованные выводы о применимости транзакционного подхода, пакетной обработки и потоковой CDC-репликации в системах с требованиями, близкими к условиям рассматриваемого корпоративного проекта.

1.5 Выводы по разделу

Таким образом, в первой главе была рассмотрена задача передачи изменений данных из операционного контура во внешний аналитический контур на примере проекта ЛКС. Были описаны три подхода, применявшиеся для решения этой задачи: Transactional Outbox, пакетная обработка на основе Spring Batch и CDC-репликация с использованием Debezium. Поскольку данные подходы по-разному организуют фиксацию, извлечение и передачу изменений, их дальнейшее сравнение в единых условиях необходимо для оценки практической применимости в рамках исследуемой системы.

2. ПРОЕКТНЫЙ РАЗДЕЛ

2.1 Постановка задачи проектирования решений для передачи изменений данных

На основе результатов исследовательского раздела в рамках данной работы ставится задача проектирования решений для передачи изменений данных из операционного контура в аналитический с последующей реализацией и сравнительным анализом трёх подходов: транзакционного подхода на основе паттерна Outbox, пакетной обработки на основе Spring Batch и потоковой CDC-репликации на основе Debezium. Для обеспечения корректного сопоставления указанные подходы должны быть реализованы в единых условиях на основе упрощённой модели корпоративного сценария.

2.1.1 Цели и ограничения

Целью проектирования является разработка трёх сопоставимых решений для передачи изменений данных из операционного контура в аналитический и создание основы для их последующего сравнительного анализа. Указанные решения должны быть построены на подходах, использованных в корпоративном проекте: транзакционном подходе на основе паттерна Outbox, пакетной обработке на основе Spring Batch и потоковой CDC-репликации на основе Debezium. Проектирование должно обеспечить возможность их последующей реализации и корректного сравнения в единых условиях.

При проектировании необходимо обеспечить единый сценарий обработки данных для всех трёх подходов, чтобы различия в результатах определялись особенностями самих решений, а не различием предметной логики или внешних условий. Для этого требуется использовать общую модель источника данных, единый целевой контур приёма изменений и сопоставимые сценарии нагрузки.

Одним из ограничений проектирования является необходимость сохранения баланса между приближением к корпоративному проекту и допустимым уровнем упрощения. С одной стороны, разрабатываемые решения должны отражать реальную задачу передачи изменений данных в DWH. С другой стороны, в рамках исследования не требуется воспроизводить всю функциональную и инфраструктурную сложность промышленной системы. Поэтому проектируемые решения должны охватывать только те компоненты и взаимодействия, которые существенны для реализации и сравнения рассматриваемых подходов.

Дополнительным ограничением является требование воспроизводимости результатов. Архитектура решений должна быть достаточно простой и однородной, чтобы обеспечить возможность проведения серии повторяемых экспериментов и объективного сопоставления характеристик производительности. В связи с этим при проектировании исключаются второстепенные бизнес-функции и интеграционные процессы, не оказывающие прямого влияния на передачу изменений данных.

2.1.2 Упрощения относительно корпоративного проекта

Прежде всего, вместо полного микросервисного ландшафта рассматривается ограниченная архитектурная схема, включающая источник данных, механизм передачи изменений и целевой контур приёма данных. Такой уровень абстракции позволяет воспроизвести ключевой интеграционный сценарий без необходимости моделирования всех сервисов и межсервисных взаимодействий, присутствующих в промышленной системе.

Кроме того, в экспериментальной постановке используется ограниченный набор сущностей и операций над ними. В рамках исследования достаточно моделирования типовых изменений данных, таких как создание, обновление и передача записей в целевой контур. Это позволяет сохранить предметную сущность задачи, не перегружая проектирование деталями конкретной бизнес-логики корпоративного проекта.

Упрощается и целевая аналитическая система. Вместо полноценного DWH с дальнейшими процессами преобразования, агрегации и потребления данных рассматривается упрощённый контур, фиксирующий факт получения и обработки изменений. Такое решение позволяет сосредоточить внимание именно на механизме доставки данных, а не на последующей аналитической обработке.

Также в рамках проектирования не воспроизводятся дополнительные аспекты промышленной эксплуатации, включая сложные схемы отказоустойчивости, расширенные механизмы безопасности, развитую систему межсервисных зависимостей и сопутствующие интеграции, не влияющие напрямую на исследуемый процесс. Это связано с тем, что целью работы является не моделирование полного корпоративного окружения, а разработка сопоставимых решений для передачи изменений данных в контролируемых условиях.

2.1.3 Критерии сравнения подходов

Для последующего сопоставления разработанных решений необходимо определить критерии сравнения и соответствующие им измеримые метрики. Поскольку в рамках данной работы проводится сравнительный анализ характеристик производительности подходов к передаче изменений данных, в качестве основы сравнения используются только те показатели, которые могут быть зафиксированы в ходе эксперимента количественно. При этом выбранные критерии соотносятся с требованиями к доставке данных, сформулированными в исследовательском разделе, прежде всего с требованиями к масштабируемости, влиянию на основную систему и устойчивости работы интеграционного контура.

Первым критерием является влияние подхода на сервис-источник данных. Для его оценки используются метрики загрузки процессора, потребления памяти и характеристик ввода-вывода. В рамках исследования анализируются показатели CPU usage, CPU load, JVM heap memory, JVM non-heap memory, а

также метрики подсистемы ввода-вывода: I/O rate, I/O errors, I/O duration и I/O utilization. Эти показатели позволяют оценить, насколько реализация каждого подхода влияет на вычислительные и системные ресурсы сервиса, в котором формируются изменения данных, и тем самым определить допустимость его применения в продакшн среде.

Вторым критерием является нагрузка на инфраструктурные компоненты передачи данных, прежде всего на брокер сообщений Kafka. Для его оценки используются метрики messages per second, messages per minute и lag by consumer group. Эти показатели позволяют определить интенсивность публикации сообщений, динамику поступления нагрузки на брокер и степень отставания потребителей от входящего потока. Тем самым данный критерий позволяет оценить, насколько подход сохраняет работоспособность при увеличении объёма передаваемых данных и соответствует требованиям к масштабируемости интеграционного решения.

Третьим критерием является нагрузка на базу данных источника. Для её оценки используются метрики, отражающие транзакционную активность, интенсивность операций чтения и записи, а также конкурентный доступ к данным. В рамках исследования анализируются показатели числа активных соединений с базой данных, количества зафиксированных и откатанных транзакций, интенсивности операций insert, update и delete, числа считанных и возвращённых строк, коэффициента попадания в буферный кэш, количества блокировок различных типов. Данные метрики позволяют определить, насколько выбранный подход увеличивает нагрузку на СУБД, влияет на работу транзакционного контура и создаёт дополнительные издержки для основной базы данных.

Таким образом, выбранные критерии сравнения отражают не только характеристики производительности, но и более общие требования к доставке данных, выявленные в исследовательском разделе. Влияние на сервис-источник позволяет оценить воздействие подхода на основную систему, нагрузка на базу данных источника - степень влияния на транзакционную и хранилищную

подсистему, а нагрузка на Kafka - устойчивость и масштабируемость интеграционного контура при росте нагрузки.

2.2 Проектирование транзакционного подхода на основе паттерна Outbox

В рамках транзакционного подхода на основе паттерна Outbox передача изменений данных из операционного контура во внешний аналитический организуется через промежуточную фиксацию событий в базе данных источника. Архитектура решения включает сервис-источник данных, основную базу данных, брокер сообщений, сервис-потребитель и внешний аналитический контур.

Источником изменений в системе выступает сервис rap-appeals, отвечающий за обработку обращений клиентов. В процессе выполнения бизнес-операций сервис сохраняет и изменяет данные в основной базе данных PostgreSQL. Особенностью данного подхода является то, что одновременно с изменением бизнес-сущностей в той же базе данных формируются записи в таблице outbox, содержащие сведения о событиях, подлежащих последующей передаче.

Дальнейшая обработка выполняется сервисом rap-dwh, который получает сообщения из брокера Kafka, преобразует данные и подготавливает их к загрузке во внешний аналитический контур. Помимо переноса бизнес-полей, сервис дополняет каждую запись служебными атрибутами, используемыми в целевой системе: идентификатором события (EVENT_GUID\$), временем фиксации изменения в источнике (SOURCE_CHANGE_TIME\$), временем загрузки в целевой контур (TARGET_CHANGE_TIME\$), последовательным идентификатором записи (TARGET_SEQUENCE_ID\$) и признаком типа операции (DML_FLG\$). После преобразования и обогащения данные записываются в целевую базу данных Oracle DB.

При этом Oracle DB и использующая её аналитическая система не входят в состав системы и рассматриваются как внешний аналитический контур. Архитектурная схема реализации [15] транзакционного подхода на основе паттерна Outbox представлена на рисунке 2.1.

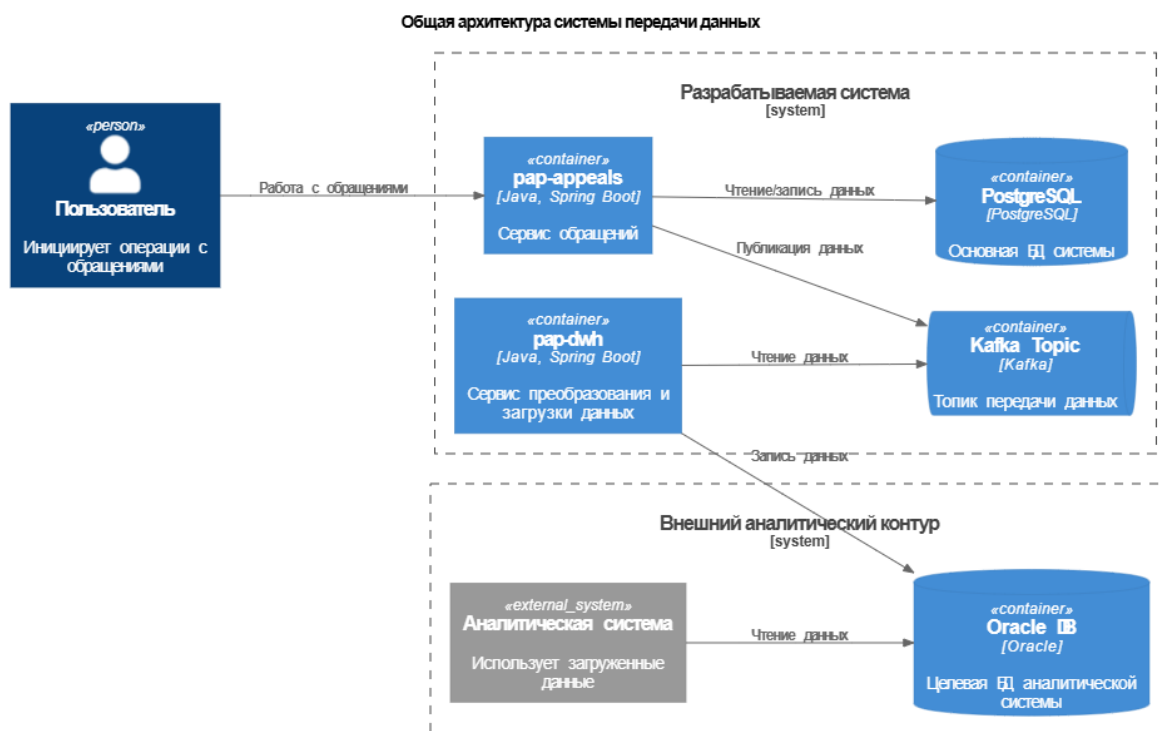


Рисунок 2.1 – Архитектурная схема реализации транзакционного подхода [разработано автором]

Принцип работы транзакционного подхода на основе паттерна Outbox основан на разделении процесса фиксации бизнес-изменений и их последующей передачи во внешний контур на два взаимосвязанных этапа. На первом этапе в рамках выполнения пользовательской или системной бизнес-операции сервис rap-appeals открывает транзакцию, сохраняет изменения в основной таблице предметной области и в той же транзакции формирует запись в таблице outbox, содержащую данные о событии, подлежащем отправке. После успешного завершения транзакции фиксируются как изменения бизнес-сущности, так и информация, необходимая для дальнейшей передачи данных.

На втором этапе отдельный фоновый процесс выполняет периодическое чтение необработанных записей из таблицы outbox. Полученные события преобразуются в сообщения для публикации в Kafka, после чего соответствующие записи помечаются как отправленные. Далее сервис pap-dwh считывает сообщения из брокера, выполняет преобразование данных, дополняет их служебными атрибутами целевой системы и загружает результат в Oracle DB. Такая организация позволяет отделить бизнес-транзакцию от асинхронной доставки данных и обеспечить поэтапную передачу изменений во внешний аналитический контур.

Последовательность взаимодействия [16] компонентов при реализации паттерна Outbox представлена на рисунке 2.2.

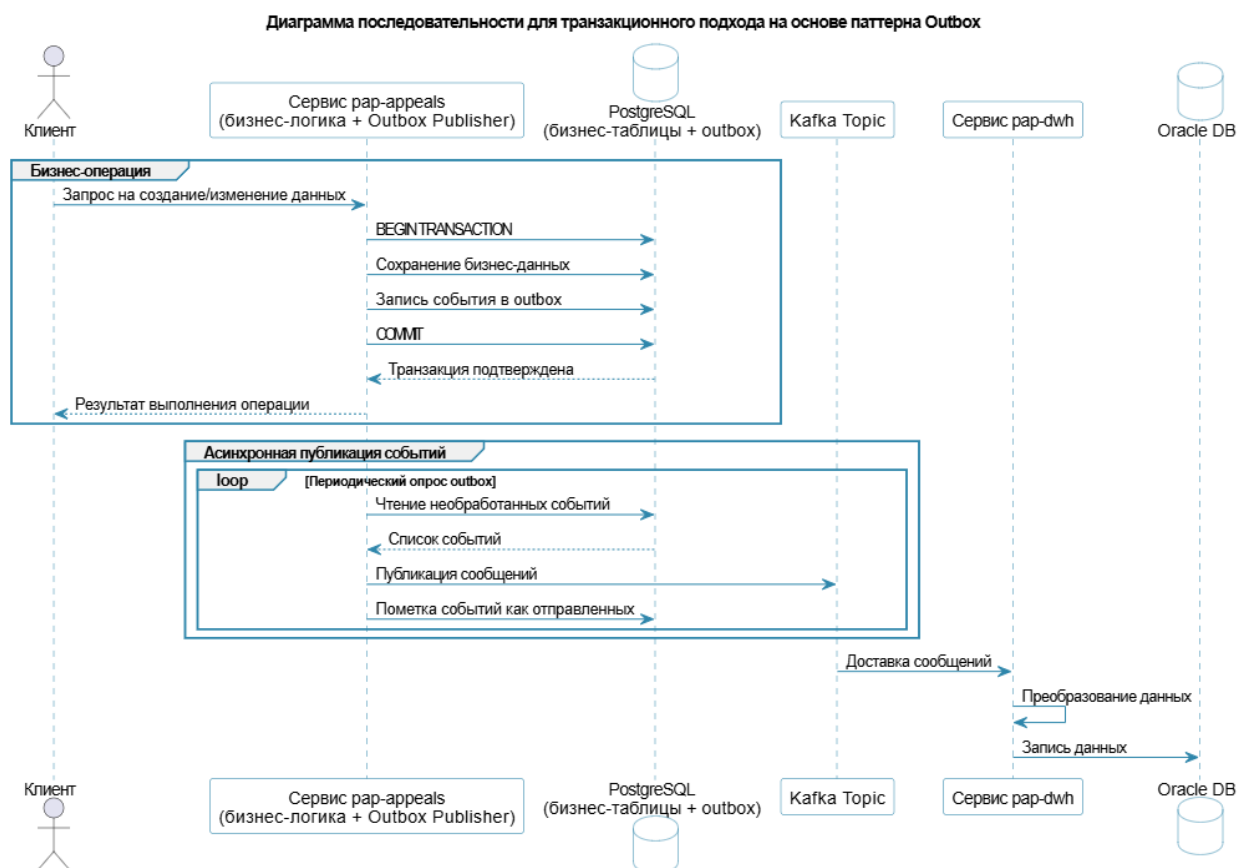


Рисунок 2.2 - Последовательность взаимодействия компонентов при реализации паттерна Outbox [разработано автором]

Для описания структуры хранения данных, используемой в рамках транзакционного подхода, построена ER-диаграмма [17]. Она отражает основные сущности предметной области, связи между ними, а также таблицу

outbox, применяемую для фиксации событий и их последующей передачи во внешний аналитический контур (Рисунок 2.3).

Рисунок 2.3 – ER-диаграмма транзакционного подхода на основе паттерна Outbox
[разработано автором]

Таблица shedlock [18-19] применяется для организации распределённой блокировки фонового процесса, выполняющего чтение записей из outbox и их последующую отправку. Использование такой таблицы позволяет гарантировать, что в каждый момент времени обработку очереди событий

выполняет только один экземпляр фонового процесса, что важно при запуске сервиса в нескольких экземплярах.

2.3 Проектирование пакетной обработки на основе Spring Batch

В рамках данного подхода архитектура решения в целом не изменяется по сравнению с транзакционным подходом. В передаче данных продолжают участвовать те же основные компоненты: сервис rap-appeals, база данных PostgreSQL, брокер сообщений Kafka, сервис rap-dwh, а также внешний аналитический контур, включающий Oracle DB и аналитическую систему. Как и в предыдущем случае, сервис rap-appeals выступает источником изменений данных, а сервис rap-dwh выполняет чтение сообщений из брокера, их преобразование и последующую загрузку в целевую базу данных.

Отличие данного решения заключается не в составе компонентов, а в способе организации передачи данных. Если в транзакционном подходе событие фиксируется одновременно с бизнес-изменением и затем асинхронно публикуется из таблицы outbox, то в пакетном подходе передача организуется в рамках отдельного процесса по расписанию. Изменения накапливаются в основной базе данных, после чего по расписанию запускается batch-процесс, который считывает необходимые записи, формирует сообщения и отправляет их в Kafka для последующей обработки сервисом rap-dwh (Рисунок 2.4).

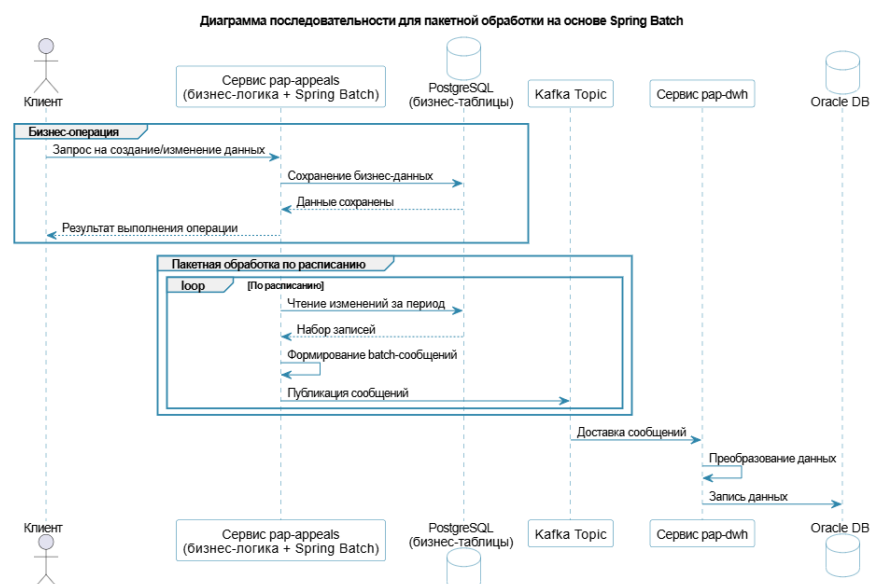


Рисунок 2.4 – Диаграмма последовательности для пакетной обработки на основе Spring Batch [разработано автором]

С точки зрения предметной области пакетный подход использует ту же основную модель данных, что и транзакционный подход, поскольку обработка по-прежнему выполняется над сущностями, связанными с обращениями клиентов. Отличие заключается в том, что вместо таблиц outbox и shedlock, применявшихся для асинхронной публикации событий, в системе используются служебные таблицы Spring Batch, предназначенные для управления пакетными заданиями и хранения информации об их выполнении. ER-диаграмма модели данных для пакетной обработки представлена на рисунке 2.5.

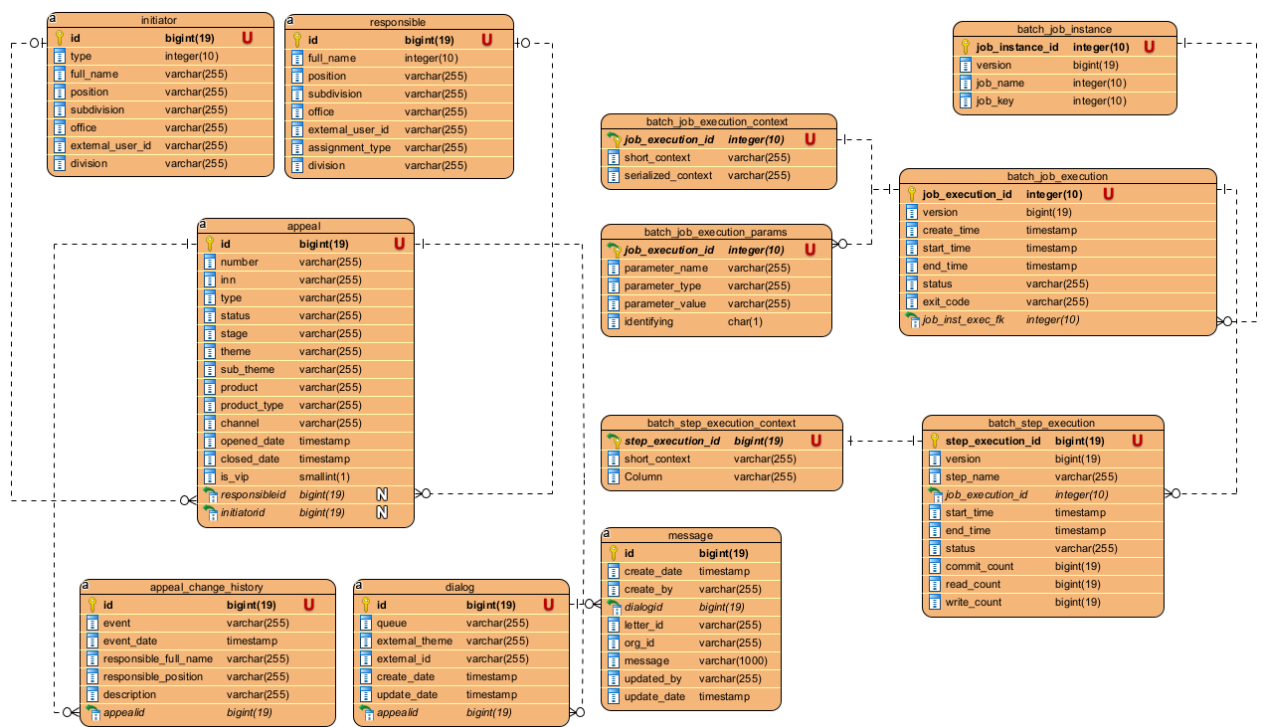


Рисунок 2.5 – ER диаграмма для пакетной обработки на основе Spring Batch
[разработано автором]

Служебные таблицы Spring Batch обеспечивают управление пакетной обработкой и хранение информации о её выполнении. BATCH_JOB_INSTANCE и BATCH_JOB_EXECUTION используются для учёта экземпляров заданий и их запусков, BATCH_JOB_EXECUTION_PARAMS - для хранения параметров запуска, BATCH_STEP_EXECUTION - для фиксации выполнения отдельных шагов batch-процесса, а таблицы BATCH_JOB_EXECUTION_CONTEXT и BATCH_STEP_EXECUTION_CONTEXT - для сохранения контекста обработки, необходимого для контроля состояния и повторного запуска задания.

2.4 Проектирование потоковой CDC-репликации на основе Debezium

В рамках данного подхода передача изменений данных организуется иначе, чем в транзакционном и пакетном решениях. Источником изменений по-прежнему выступает сервис rap-appeals, выполняющий операции с обращениями клиентов и сохраняющий данные в PostgreSQL, однако дальнейшая передача информации во внешний аналитический контур

осуществляется не через сервис rap-dwh, а напрямую с использованием CDC-механизма. Архитектурная схема реализации потоковой CDC-репликации на основе Debezium представлена на рисунке 2.6

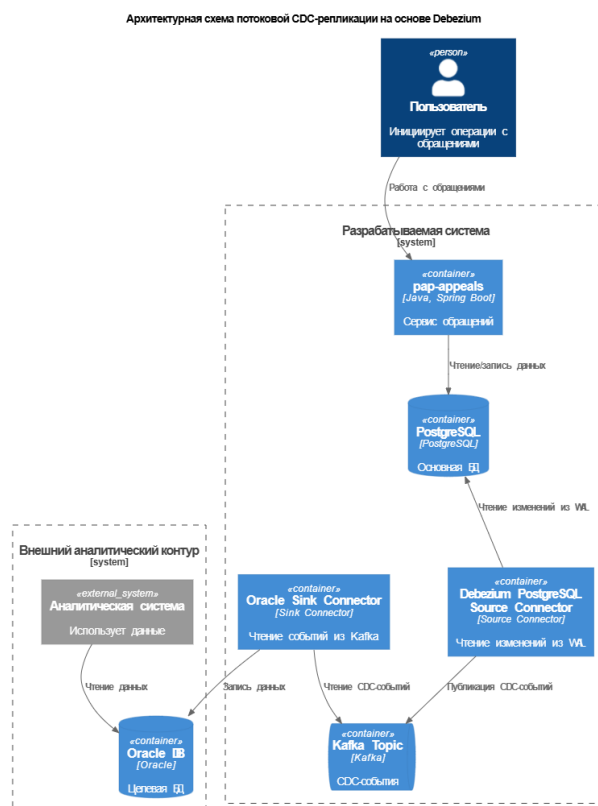


Рисунок 2.6 - Архитектурная схема реализации потоковой CDC-репликации на основе Debezium [разработано автором]

Основой решения является технология Debezium, которая отслеживает изменения в таблицах базы данных PostgreSQL на уровне журнала транзакций. Зафиксированные изменения передаются в поток сообщений, после чего с использованием коннекторов [20-23] загружаются в целевую базу данных Oracle DB. Последовательность взаимодействия компонентов при реализации потоковой CDC-репликации представлена на рисунке 2.7.

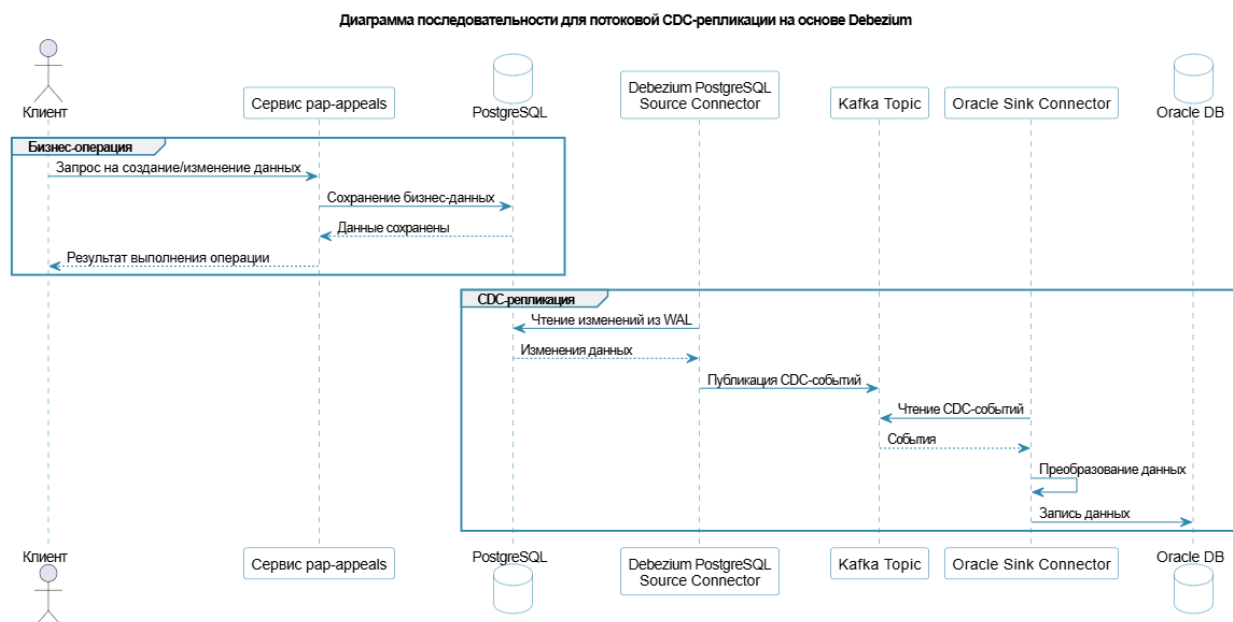


Рисунок 2.7 - Последовательность взаимодействия компонентов при реализации потоковой CDC-репликации [разработано автором]

Таким образом, в данном варианте прикладной сервис не формирует отдельные сообщения на уровне бизнес-логики и не запускает выгрузку данных, а извлечение изменений переносится на инфраструктурный уровень.

2.5 Выводы по разделу

В рамках проектного раздела сформулирована задача проектирования решений для передачи изменений данных и определены критерии их последующего сравнения. Для трёх рассматриваемых подходов описаны архитектура, логика работы и модель данных.

В результате сформированы три сопоставимых решения, которые служат основой для их последующей реализации и экспериментального сравнения в следующем разделе.

3. ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

В данном разделе рассматривается практическая реализация подходов к передаче изменений данных в рамках исследуемого проекта. Основное внимание уделяется вопросам разработки программных компонентов, обеспечивающих работу транзакционного, пакетного и потокового подходов, а также последующему экспериментальному исследованию.

3.1 Реализация транзакционного подхода на основе паттерна Outbox

В рамках проекта транзакционный подход был реализован как один из вариантов передачи изменений данных для последующего тестирования и сравнительного анализа. Реализация включает программные компоненты, отвечающие за фиксацию изменений бизнес-сущностей, накопление идентификаторов изменённых объектов в рамках транзакции, подготовку данных для выгрузки и формирование записей в таблице outbox. Далее рассматриваются основные элементы реализации данного решения.

Для фиксации изменений сущности был разработан слушатель JPA-событий `AppealEntityListener` [24-25]. Он вызывается после создания и обновления экземпляров сущности `AppealEntity` и используется как точка входа в механизм синхронизации (Приложение А.1).

Приведённый листинг показывает, что после сохранения (`@PostPersist`) или обновления (`@PostUpdate`) сущности вызывается метод `postPersistOrUpdate`. Внутри данного метода не выполняется непосредственная запись в таблицу outbox. Вместо этого вызываются два метода сервиса `DwhService`: `registerSynchronization()` и `addToOutbox(...)`.

Первый метод регистрирует обработчик, который будет выполнен перед фиксацией транзакции. Второй метод добавляет идентификатор изменённой сущности в специальный набор, связанный с текущим транзакционным

контекстом. Такое решение позволяет не формировать сообщения немедленно при каждом изменении объекта, а отложить подготовку данных до момента завершения всей бизнес-операции. Это особенно важно в случаях, когда в рамках одной транзакции объект изменяется несколько раз: в outbox будет отправлено не несколько промежуточных состояний, а итоговое актуальное состояние сущности.

Логика накопления идентификаторов и подготовки данных к выгрузке реализована в сервисе DwhService (Приложение А.2).

В данной реализации используется механизм TransactionSynchronizationManager [26], позволяющий привязать дополнительные данные и действия к текущей транзакции. Это решение было выбрано для того, чтобы выполнить формирование payload непосредственно перед подтверждением транзакции, когда состояние изменённых сущностей уже окончательно сформировано.

Метод addToOutbox отвечает за накопление идентификаторов обращений. В качестве хранилища используется ресурс транзакции, доступный по ключу RESOURCE_KEY. Если в рамках текущей транзакции набор идентификаторов ещё не создан, он инициализируется как LinkedHashSet, после чего привязывается к контексту транзакции. Использование именно множества позволяет исключить дублирование идентификаторов. Это особенно важно, если одна и та же сущность модифицируется несколько раз в рамках одного бизнес-сценария.

Метод registerSynchronization регистрирует обработчик transactionSynchronizationAdapter. Наиболее важная часть логики находится внутри переопределённого метода beforeCommit. Он выполняется непосредственно перед фиксацией транзакции. В этот момент из контекста транзакции извлекается набор идентификаторов изменённых обращений, после чего выполняется их повторное чтение из базы данных через appealService.findAllById(ids).

Повторное чтение сущностей необходимо по двум причинам. Во-первых, это позволяет работать с итоговым состоянием объектов, а не с промежуточным состоянием, полученным в момент вызова слушателя. Во-вторых, это упрощает подготовку полноценного payload, так как можно сразу догрузить все дополнительные данные, необходимые для выгрузки. В рассматриваемой реализации для каждого обращения дополнительно загружается история изменений через `changeHistoryService.findAllByAppealId(...)`. После этого сущности преобразуются в DTO-объекты `AppealDwhDTO` с помощью `appealMapper`.

Полученный список DTO передаётся в `outboxService.processAppealDwh(payload)`, где уже осуществляется непосредственное формирование записей в outbox-таблице (Приложение А.3). Таким образом, `DwhService` играет роль промежуточного слоя между событиями изменения бизнес-сущностей и механизмом сохранения интеграционных сообщений.

В приведённом методе реализуется преобразование набора DTO-объектов в записи таблицы outbox. Для каждого элемента `AppealDwhDTO` создаётся объект `OutboxEntity`, содержащий три основных поля: ключ сообщения, название целевого топика и сериализованный payload.

Поле `messageKey` формируется на основе идентификатора обращения. Это необходимо для дальнейшей маршрутизации и упорядочивания сообщений на стороне брокера или потребителя. Поле `topic` задаёт логический канал доставки сообщения и извлекается из конфигурации приложения. Поле `payload` содержит сериализованную структуру `OutboxDwhDTO`, внутрь которой помещается полезная нагрузка `PayloadDwhDTO` с данными обращения.

Следующим элементом реализации транзакционного подхода является механизм вычитывания записей из таблицы outbox и их последующей обработки. Для этого в проекте был разработан отдельный фоновый процесс `OutboxScannerJob`, который запускается по расписанию [27-28] и последовательно обрабатывает накопленные записи (Приложение А.4).

Приведённый листинг показывает, что доставка сообщений из `outbox` реализована как периодически запускаемая задача. Метод `proceed()` вызывается по расписанию, параметры которого задаются через конфигурацию `initialDelayMs` и `fixedDelayMs`. Таким образом, приложение с определённой частотой выполняет проверку таблицы `outbox` на наличие необработанных записей.

Дополнительно в реализации используется аннотация `@SchedulerLock`, предоставляемая библиотекой `ShedLock`. Её использование необходимо для предотвращения одновременного запуска одной и той же задачи на нескольких экземплярах приложения. Это особенно важно для распределённой среды, в которой несколько инстансов сервиса могут работать параллельно. В отсутствие блокировки одна и та же `outbox`-запись могла бы быть обработана несколько раз разными экземплярами приложения.

Внутри метода `proceed()` реализован цикл, который продолжает работу до тех пор, пока в таблице `outbox` находятся данные для обработки. На каждой итерации вызывается метод `outboxService.findElements()`, возвращающий очередную порцию записей. Если записи найдены, для каждой из них подбирается соответствующий обработчик из списка `outboxEventHandlers`.

Такой способ организации даёт реализации дополнительную гибкость. Вместо жёсткой привязки всей логики доставки к одному типу сообщений используется набор обработчиков, каждый из которых может определить применимость к конкретной `outbox`-записи с помощью метода `isApplicable(...)`. После выбора подходящего обработчика вызывается метод `handle(outboxEntity)`, в котором выполняется собственно доставка или обработка сообщения.

После успешной обработки запись удаляется из таблицы `outbox` методом `outboxService.delete(outboxEntity.getId())`. Тем самым реализуется стандартная схема работы с `outbox`-таблицей: запись существует в базе данных до тех пор, пока не будет гарантированно обработана фоновым процессом. Такой подход повышает надёжность решения, поскольку сообщение не теряется при

временной недоступности внешнего брокера или обработчика: оно остаётся в таблице до следующей итерации фоновой задачи.

После публикации сообщения из outbox во внешний брокер дальнейшая обработка выполняется в отдельном сервисе pap-dwh, который отвечает за приём интеграционных событий и их сохранение в целевую Oracle-базу хранилища данных. Таким образом, транзакционный подход в рамках рассматриваемого проекта не ограничивается только формированием outbox-записей в сервисе-источнике, а включает также отдельный контур доставки и загрузки данных в DWH.

Приём сообщений в сервисе pap-dwh реализован через Kafka listener [29]. После получения сообщения выполняется проверка флага конфигурации, позволяющего при необходимости отключить обработку, после чего сообщение десериализуется и передаётся в сервис сохранения (Листинг 3.1).

Листинг 3.1 – Приём сообщения в сервисе pap-dwh [разработано автором]

```
@SneakyThrows
private void handleAppealsRequest(String json) {
    if (!kafkaListenerProperties.isAppealsProcessingEnabled()) {
        log.info("Processing appeals is disabled");
        return;
    }
    log.trace("Received kafka message from pap-appeals-sme: {}", json);
    AppealOutboxPayloadDTO payload = objectMapper.readValue(json,
        AppealOutboxPayloadDTO.class);
    appealOutboxService.save(payload);
}
```

Данный фрагмент показывает, что сервис pap-dwh работает как потребитель интеграционных сообщений. На вход он получает сериализованный JSON, который затем преобразуется в объект AppealOutboxPayloadDTO. После десериализации данные передаются в прикладной сервис appealOutboxService, где начинается их обработка с точки зрения целевой модели хранилища данных.

Следующим уровнем реализации является слой репозитория, отвечающий за вставку данных в Oracle. Для унификации механизма сохранения был разработан абстрактный базовый репозиторий AbstractDwhRepository, содержащий общую логику подготовки служебных полей, преобразования сущностей в набор SQL-параметров и выполнения вставки (Приложение А.5).

В данном листинге видно, что в базовом репозитории выделены две группы параметров. Первая группа – это служебные поля DWH-слоя, общие для всех сохраняемых сущностей.

Вторая группа параметров определяется уже конкретной доменной сущностью и добавляется в специализированных репозиториях через метод `addTableParameters(...)`. Такая архитектура позволяет централизовать общую логику загрузки в хранилище и одновременно сохранить возможность расширения для разных типов бизнес-объектов.

Для сохранения данных по обращениям используется конкретная реализация `AppealJdbcRepository` (Приложение А.6).

Приведённая реализация показывает, каким образом сообщение, изначально сформированное в сервисе-источнике, трансформируется в строку целевой таблицы Oracle. Репозиторий `AppealJdbcRepository` определяет имя таблицы назначения и набор столбцов, в которые мапятся поля объекта `AppealDTO`. Тем самым сервис `rap-dwh` выполняет уже не просто роль промежуточного потребителя сообщения, а роль адаптера между интеграционным форматом и структурой корпоративного хранилища данных.

3.2 Реализация пакетной обработки на основе Spring Batch

В качестве второго варианта передачи изменений данных в рамках проекта был реализован пакетный подход на основе `Spring Batch`. В отличие от транзакционного решения, где формирование интеграционного сообщения инициировалось непосредственно при изменении бизнес-сущности, здесь обработка была вынесена в отдельный batch-процесс. Такой подход позволил отделить контур передачи данных от основного пользовательского потока и снизить влияние механизма синхронизации на транзакционную логику прикладного сервиса.

Разработка пакетного решения была ориентирована на периодическое чтение изменённых записей из базы данных, их последовательную обработку и

подготовку к передаче во внешний контур. При этом в рамках данного подраздела рассматривается именно реализация batch-механизма на стороне сервиса-источника. Дальнейшая отправка данных выполняется в сервис rap-dwh и обрабатывается далее по той же схеме, что и в транзакционном подходе.

Основу реализации составляет конфигурация batch-job, в которой определены задание, шаг и компонент чтения данных из репозитория (Приложение Б.1).

В приведённой конфигурации определяется задание migrateAppealJob, которое состоит из одного шага migrateAppealStep. Для идентификации отдельных запусков используется RunIdIncrementer, а параметр preventRestart() запрещает автоматический перезапуск уже завершённого задания с теми же параметрами. Такое решение было выбрано для того, чтобы каждый запуск batch-задачи рассматривался как отдельный цикл обработки данных.

Ключевой частью реализации является шаг migrateAppealStep, построенный по chunk-модели. В данном случае обрабатываемые элементы имеют тип AppealEntity, а результат преобразования – AppealDwhDTO. Размер чанка извлекается из конфигурации и определяет количество записей, обрабатываемых в рамках одной транзакции Spring Batch. Использование chunk-подхода позволяет балансировать между производительностью и нагрузкой на систему: увеличение размера чанка уменьшает число транзакционных переключений, но при этом увеличивает объём данных, удерживаемых в памяти на одной итерации.

Чтение данных реализовано через RepositoryItemReader, который строится поверх AppealRepository. Такой выбор упрощает интеграцию с уже существующим слоем доступа к данным и позволяет использовать готовые методы репозитория без написания отдельного JDBC-кода для batch-чтения. Дополнительно задаётся сортировка по полю lastUpdateDate в возрастающем порядке. Это обеспечивает стабильный и воспроизводимый порядок чтения записей, что важно как для корректности обработки, так и для сопоставимости результатов в экспериментальной части работы.

Существенной особенностью реализации является использование `@StepScope` и `job`-параметров. Это позволяет конфигурировать поведение `batch`-задания в зависимости от режима запуска. В рассматриваемой реализации поддерживаются два режима: `DAILY` и `MIGRATION`. Логика выбора режима вынесена в метод `setReaderRepositoryMethod(...)`.

При режиме `DAILY` используются два временных интервала: один для открытых обращений и один для закрытых. Это позволяет учитывать различную специфику изменения данных по этим категориям объектов. Для открытых обращений обычно требуется более короткий интервал актуализации, тогда как закрытые обращения можно перечитывать за более длинный период. Для этого из параметров задания извлекаются значения интервалов в днях, после чего формируются границы временных окон и вызывается репозиторный метод `findAllInTwoIntervals(...)`.

При режиме `MIGRATION` используется единый широкий интервал, предназначенный для полной или расширенной выгрузки данных. В этом случае вызывается метод `findAllInIntervals(...)`, который возвращает записи, изменённые за заданный период. Такой режим может использоваться как при первичной загрузке, так и при необходимости повторной синхронизации значительного объёма данных.

Тем самым в ходе разработки `batch`-решения была обеспечена возможность работы как в режиме регулярной инкрементальной обработки, так и в режиме расширенной миграции. Это делает решение более гибким и пригодным для разных эксплуатационных сценариев.

Параметры `batch`-задания вынесены в отдельный класс конфигурации, что позволяет настраивать поведение процесса без изменения прикладного кода (Листинг 3.2).

Листинг 3.2 – Параметры batch-обработки [разработано автором]

```
@Data
@ConfigurationProperties(value = "pap-appeals-sme.batch")
public class BatchProperties {
    private int corePoolSize = 10;
    private int maxPoolSize = 10;
    private String threadNamePrefix = "appeals-executor-";
    private final MigrateAppealsJob migrateAppealsJob = new MigrateAppealsJob();
    @Data
    public static class MigrateAppealsJob {
        private int chunkSize = 100;
        private int pageSize = 100;
        private int lastUpdateDateIntervalForAppealsInDays = 10;
        private int lastUpdateDateIntervalForClosedAppealsInDays = 30;
        private int lastUpdateDateIntervalForFullMigrationInDays = 365;
    }
}
```

Вынесение параметров в `BatchProperties` решает сразу несколько задач. Во-первых, это позволяет гибко изменять размер чанка и страницы чтения, не перекомпилируя приложение. Во-вторых, становится возможным отдельно регулировать интервалы выборки данных для разных режимов обработки. В-третьих, конфигурация потоков выполнения также выведена на уровень параметров, что создаёт основу для масштабирования batch-задания при необходимости.

Параметр `chunkSize` определяет объём данных, обрабатываемых за одну batch-транзакцию, тогда как `pageSize` управляет размером страницы при чтении данных из репозитория. Эти два параметра критичны для последующего тестирования, так как напрямую влияют на нагрузку на приложение, базу данных и подсистему памяти. Интервалы `lastUpdateDateIntervalForAppealsInDays`, `lastUpdateDateIntervalForClosedAppealsInDays` и `lastUpdateDateIntervalForFullMigrationInDays` задают временные окна выборки и тем самым определяют объём данных, попадающих в конкретный batch-запуск.

3.3 Реализация потоковой CDC-репликации на основе Debezium

В качестве третьего варианта передачи изменений данных в рамках проекта была реализована потоковая CDC-репликация на основе Debezium и

Kafka Connect. В отличие от транзакционного и пакетного подходов, данное решение не требует внесения изменений в прикладные сервисы-источники. Логика формирования и передачи изменений полностью выносится на уровень инфраструктуры базы данных и платформы потоковой интеграции. Это означает, что основные сервисы продолжают работать без дополнительного кода, связанного с регистрацией изменений, формированием outbox-сообщений или запуском batch-процессов.

С практической точки зрения это является ключевым отличием данного подхода от двух предыдущих решений. Если при использовании паттерна Outbox и Spring Batch источник данных сам участвовал в подготовке интеграционных сообщений, то в случае CDC-репликации сервис-источник остаётся неизменным, а извлечение изменений выполняется за счёт чтения журнала транзакций PostgreSQL. Также отпадает необходимость в отдельном промежуточном сервисе rap-dwh, поскольку доставка изменений в Oracle может быть организована напрямую через Kafka Connect и JDBC Sink Connector.

Для работы данного решения в инфраструктуре был поднят отдельный контейнер kafka-connect, обеспечивающий запуск source- и sink-коннекторов (Приложение В.1).

Приведённая конфигурация показывает, что для реализации потоковой репликации потребовалось развернуть отдельный экземпляр Kafka Connect. Он подключается к Kafka-кластеру, использует собственные служебные топики для хранения конфигурации, смещений и статусов, а также предоставляет REST-интерфейс для регистрации и управления коннекторами.

Особое значение в данной конфигурации имеют каталоги с подключаемыми расширениями. Для работы решения потребовалось установить плагины Kafka Connect: Debezium PostgreSQL Source Connector для чтения изменений из PostgreSQL и Debezium JDBC Sink Connector для записи данных в Oracle. Кроме того, для работы sink-коннектора был подключён JDBC-драйвер Oracle (ojdbc8.jar). Таким образом, по сравнению с предыдущими подходами,

основная сложность реализации была перенесена из прикладного кода в инфраструктурную конфигурацию.

После запуска контейнера в Kafka Connect были импортированы конфигурации коннекторов. Первый коннектор отвечает за чтение изменений из PostgreSQL (Листинг В.2).

В данной конфигурации source-коннектор подключается к PostgreSQL и отслеживает изменения только в таблице `public.dialog`. Для чтения WAL используется плагин `pgoutput`, а изменения публикуются в Kafka-топик с префиксом `pg_server`. Параметры `publication.name` и `slot.name` задают логическую репликацию на стороне PostgreSQL.

Отдельного внимания заслуживает блок `transforms.unwrap`. В нём используется трансформация `ExtractNewRecordState`, которая позволяет извлекать из Debezium-события только актуальное состояние записи и одновременно добавлять служебные поля: тип операции, имя таблицы, метку времени источника, позицию в WAL и время формирования сообщения. Это упрощает последующую запись в целевую систему, поскольку sink-коннектор получает уже “плоскую” структуру данных, пригодную для дальнейшего преобразования.

Параметр `snapshot.mode = initial` означает, что при первом запуске коннектор выполняет начальный снимок таблицы, а затем переключается на потоковое чтение новых изменений. Таким образом, реализуется не только доставка новых событий, но и первичная загрузка уже существующих данных.

Второй коннектор используется для записи изменений из Kafka в Oracle (Приложение В.3).

Таким образом, если в предыдущих двух решениях преобразование данных выполнялось в Java-коде прикладных сервисов, то здесь оно полностью описывается конфигурацией коннектора. Это является важной особенностью разработанного решения: объём прикладного кода уменьшается, однако существенно возрастает роль инфраструктурной настройки и корректности конфигурации Kafka Connect.

После подготовки контейнера и установки необходимых плагинов конфигурации коннекторов импортируются в Kafka Connect через REST API. Тем самым в эксплуатационном контуре необходимо не только развернуть сам контейнер kafka-connect, но и выполнить регистрацию source- и sink-коннекторов с соответствующими JSON-конфигурациями. Только после этого система начинает автоматически считывать изменения из PostgreSQL и записывать их в Oracle.

С точки зрения разработки данное решение принципиально отличается от двух предыдущих подходов. Изменения в основные прикладные сервисы не вносятся вовсе: не требуется ни слушателей сущностей, ни outbox-таблиц, ни batch-задач. Весь механизм синхронизации данных переносится в инфраструктурный слой. Это уменьшает связанность между прикладной логикой и интеграционным контуром, однако повышает требования к инфраструктуре и её конфигурации.

3.4 Методика проведения эксперимента

Экспериментальное исследование направлено на количественное сравнение трёх реализованных подходов передачи изменений данных: транзакционного подхода на основе паттерна Outbox, пакетной обработки на основе Spring Batch и потоковой CDC-репликации на основе Debezium. Цель эксперимента заключается в определении степени влияния каждого подхода на сервис-источник данных, базу данных источника и инфраструктурные компоненты передачи данных.

Для обеспечения сопоставимости результатов все измерения выполняются в одинаковой тестовой среде, при едином наборе пользовательских сценариев и одинаковом профиле нагрузки. Изменяемым фактором выступает только механизм передачи изменений данных. Это позволяет интерпретировать различия в результатах как следствие выбранного подхода, а не различий в прикладной логике системы.

В ходе эксперимента для каждого подхода последовательно выполняется нагрузочный тест, во время которого фиксируются метрики приложения, базы данных и брокера сообщений. Анализируются как средние значения показателей, так и их изменение во времени, включая моменты роста задержек, появления ошибок и накопления отставания в интеграционном контуре.

3.4.1 Тестовые сценарии

Для проведения экспериментального исследования использовался нагрузочный сценарий, реализованный с помощью инструмента k6 [32-34]. Сценарий моделирует типовую работу пользователей сервиса обработки обращений и формирует смешанную нагрузку, включающую операции чтения, создания и изменения данных. Такой подход позволяет оценить не только производительность пользовательских API, но и влияние различных механизмов передачи изменений – Transactional Outbox, Spring Batch и CDC на базе Debezium – на работу сервиса-источника, базы данных и Kafka.

В качестве модели изменения интенсивности нагрузки использовался сценарий ramping-vus, при котором количество виртуальных пользователей (VU) увеличивается постепенно. Тест начинался с нулевого количества пользователей, после чего нагрузка поэтапно возрастала до 10 VU, затем до 20 VU, далее до 35 VU. После выхода на целевой уровень нагрузка удерживалась на уровне 35 виртуальных пользователей в течение 5 минут, после чего происходило постепенное завершение теста. Общая структура сценария представлена следующим образом: разогрев системы – рост нагрузки – удержание стабильного уровня – спад нагрузки. Такая модель позволяет проверить поведение приложения не только в момент нарастания активности, но и в период продолжительной работы под постоянной нагрузкой (Приложение Г.1).

В ходе теста в пиковые моменты интенсивность HTTP-запросов составляла примерно 20–23 операции в секунду. Данное значение зависело от текущей задержки ответов сервиса и от случайной паузы между действиями виртуальных

пользователей. Между итерациями каждый виртуальный пользователь выполнял паузу длительностью от 0.8 до 2.3 секунды, что позволяло смоделировать более реалистичное пользовательское поведение, а не непрерывную отправку запросов без ожидания.

Сценарий формировал вероятностное распределение операций. Наибольшую долю составляли операции чтения: 35% запросов приходилось на поиск обращений по ИНН, а ещё 25% – на поиск с фильтрами. Это соответствует характерному профилю эксплуатации прикладных систем, где операции просмотра и поиска данных обычно выполняются чаще, чем операции изменения. Для проверки записи данных в сценарий также была включена операция создания обращения, на которую приходилось 30% всех итераций. Оставшиеся 10% занимали операции с ранее созданными обращениями: получение обращения по идентификатору, обновление обращения и привязка ИНН.

3.4.2 Настройка тестовой среды

Экспериментальное исследование проводилось в изолированной контейнерной среде, развернутой с помощью Docker Compose [30-31] (Приложение Г.2). Такой подход позволил запускать все компоненты системы в единой конфигурации и обеспечил воспроизводимость условий эксперимента для каждого из рассматриваемых вариантов передачи данных. Благодаря контейнеризации все сервисы, базы данных, брокер сообщений и компоненты мониторинга запускались как единый стенд, что снижало влияние внешних факторов на результаты измерений.

В состав тестовой среды входили сервис-источник данных rap-appeals-sme, база данных PostgreSQL, брокер сообщений Apache Kafka, сервис-получатель данных rap-dwh, целевая база данных Oracle, а также вспомогательные компоненты Zookeeper, Kafka UI и Kafka Connect. Сервис rap-appeals-sme выступал источником бизнес-изменений: в ходе нагрузочного тестирования в нём создавались, изменялись и запрашивались обращения. База

данных PostgreSQL использовалась как основное хранилище сервиса-источника, а Kafka применялась в качестве промежуточного канала передачи изменений между источником и получателем. Сервис rap-dwh принимал данные из Kafka и обеспечивал их дальнейшую загрузку в целевую базу Oracle.

Для сценария CDC в Kafka Connect были подключены плагины Debezium PostgreSQL Connector и JDBC Sink Connector. Debezium PostgreSQL Connector использовался для чтения изменений из журнала транзакций PostgreSQL и публикации этих изменений в Kafka-топики. JDBC Sink Connector, в свою очередь, применялся для последующей доставки данных из Kafka в целевое хранилище. Такая конфигурация позволила оценить поведение CDC-подхода в условиях нагрузки без необходимости вносить дополнительные операции отправки сообщений непосредственно в бизнес-код сервиса-источника.

Для наблюдения за состоянием системы использовалась связка Prometheus [36] и Grafana [37]. Метрики приложения собирались через Spring Boot Actuator [38], что позволяло отслеживать количество HTTP-запросов, длительность их выполнения, ошибки, потребление памяти JVM, загрузку CPU и число потоков. Метрики Kafka экспортировались через kafka-exporter, благодаря чему отслеживались интенсивность публикации сообщений, скорость потребления, размер отставания consumer group и состояние топиков. Метрики PostgreSQL собирались через postgres-exporter и использовались для анализа числа транзакций, операций вставки и обновления, блокировок, deadlock-событий, чтения строк и активности соединений. Дополнительно системные показатели хоста и контейнеров собирались через node-exporter.

Визуализация и анализ собранных данных выполнялись в Grafana. Для удобства анализа использовались отдельные дашборды: дашборд приложения, дашборд PostgreSQL, дашборд Kafka и дашборд JVM. Это позволило сопоставлять поведение разных уровней системы: пользовательских HTTP-запросов, внутренней работы приложения, нагрузки на базу данных и активности интеграционного контура. Такой подход был особенно важен, поскольку

исследуемые механизмы передачи изменений создают нагрузку не только на приложение, но и на базу данных, Kafka и сервис-получатель.

Перед запуском каждого эксперимента тестовая среда приводилась в согласованное состояние. Очищались тестовые и служебные данные, сбрасывалось накопленное состояние интеграционного контура, проверялось состояние Kafka-топиков и consumer group, а также фиксировались начальные значения метрик. Это позволяло проводить эксперименты в сопоставимых условиях и исключать влияние данных, оставшихся после предыдущих запусков. Благодаря этому результаты, полученные для Transactional Outbox, Spring Batch и CDC-подхода, можно было сравнивать между собой по одинаковому набору показателей.

3.5 Результаты эксперимента и анализ метрик производительности

В данном пункте представлены результаты экспериментального исследования разработанных решений. Анализ выполняется на основе метрик, собранных в ходе нагрузочного тестирования, и направлен на выявление различий между тремя подходами.

3.5.1 Результаты по сценарию нагрузки для транзакционного решения на основе паттерна Outbox

По графику интенсивности HTTP-запросов видно, что в ходе теста нагрузка на сервис постепенно росла, в пике достигая около 20–21 операций в секунду, затем некоторое время удерживалась в диапазоне 17–20 операций в секунду и к завершению теста снижалась примерно до 12 операций в секунду. При этом на графике ошибок заметных всплесков HTTP 5xx не наблюдается, то есть в данном сценарии сервис в целом сохранял работоспособность без массовых отказов (Рисунок 3.1). График длительности HTTP-запросов показывает, что среднее время ответа оставалось менее 1 секунды, а текущее

значение находилось на уровне около 492 мс. Однако максимальная длительность отдельных запросов возрастала до 6.89 секунды, что указывает на появление отдельных медленных операций при росте нагрузки.

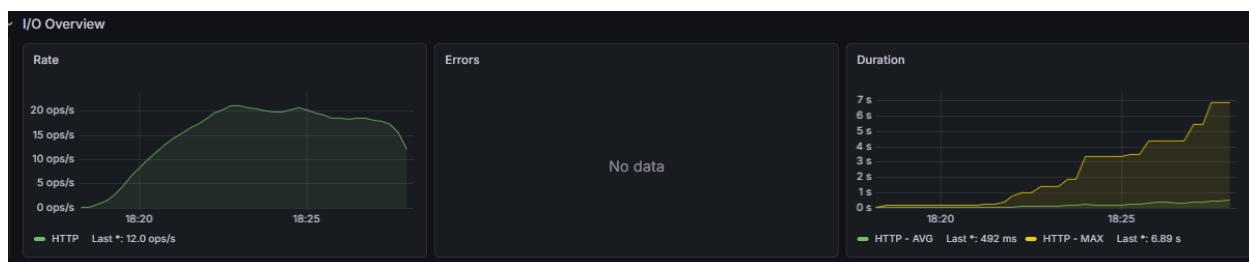


Рисунок 3.1 – Графики I/O Overview для транзакционного подхода [разработано автором]

С точки зрения использования памяти JVM критических ограничений не наблюдалось. На графиках видно, что фактическое использование heap memory составляло около 132 MiB, а максимальное значение за время теста достигало примерно 191 MiB. Общий объём используемой памяти JVM поднимался до 297 MiB при максимуме около 356 MiB, что заметно ниже доступного лимита. Показатели non-heap memory также оставались стабильными – в пределах 165–167 MiB (Рисунок 3.2).

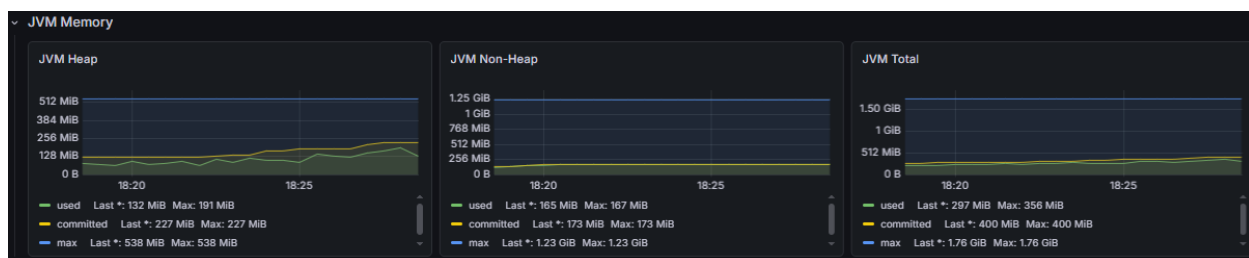


Рисунок 3.2 – Графики JVM Memory для транзакционного подхода [разработано автором]

Иная картина наблюдается по вычислительным ресурсам. На графике CPU Usage видно, что загрузка процессора в пике достигала около 55.5%, а показатель Load возрастал до 3.0 при двух доступных CPU. Одновременно число потоков увеличивалось примерно с 28–30 до 40–41, а количество daemon-потоков достигало 35. Это показывает, что при использовании Outbox приложение работает в более напряжённом режиме: основная нагрузка проявляется не по памяти, а по CPU и пулу потоков (Рисунок 3.3).

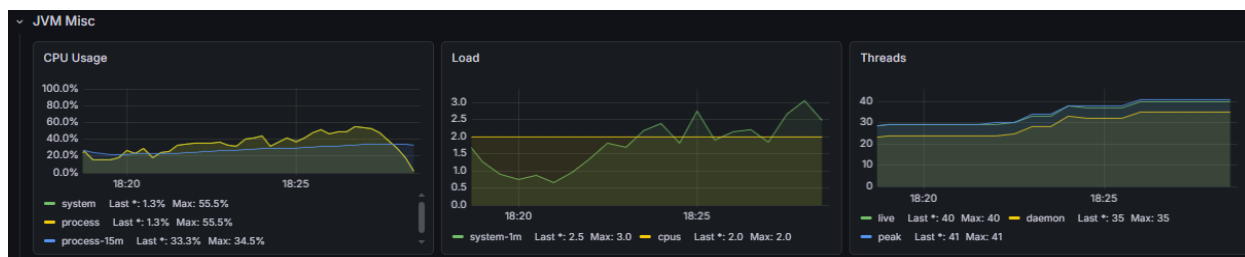


Рисунок 3.3 – Графики JVM Misc для транзакционного подхода [разработано автором]

По графикам Kafka видно, что интенсивность публикации сообщений в топик быстро возрастает и в пике достигает примерно 228 сообщений в секунду. На графике Message in per minute суммарная интенсивность публикации поднимается до 5.38 тыс. сообщений в минуту, после чего к завершению теста снижается до 373 сообщений в минуту (Рисунок 3.4). Такое поведение объясняется спецификой Transactional Outbox: в Kafka публикуются события, сформированные на основе изменений в базе данных, поэтому объём потока оказывается существенно выше, чем число самих пользовательских операций.

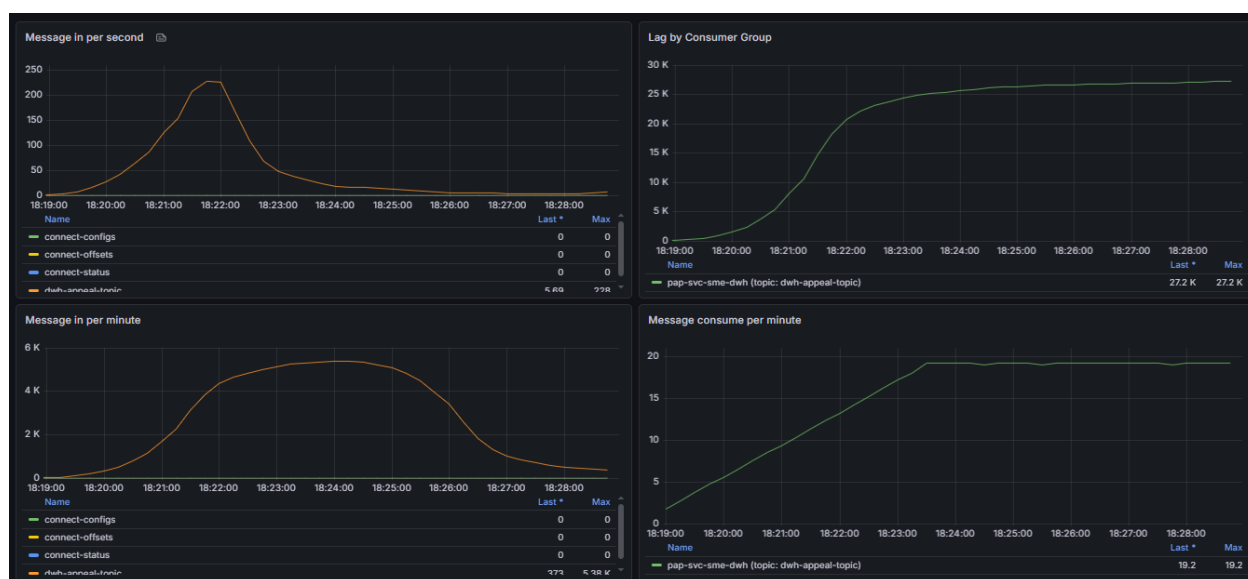


Рисунок 3.4 – Графики потока изменений в топике Kafka [разработано автором]

При этом график Lag by Consumer Group показывает устойчивый рост отставания потребителя до 27.2 тыс. сообщений. Скорость потребления по графику Message consume per minute остаётся значительно ниже и держится на уровне около 19.2 сообщений в минуту. Это означает, что потребитель не успевает обрабатывать входящий поток с той же скоростью, с которой сообщения публикуются, из-за чего в Kafka начинает накапливаться очередь.

Метрики PostgreSQL также показывают заметное усиление нагрузки на базу данных. Число commit-транзакций в пике достигало 491, тогда как rollback оставался практически на нулевом уровне. Интенсивность операций insert поднималась примерно до 677, update - до 1311, delete - до 233. Одновременно число возвращаемых строк на графике Tuples fetched/returned возрастало до 1.16 млн, что указывает на высокую активность чтения и обработки данных. При этом I/O Read/Write time оставался почти на нулевом уровне, то есть выраженного давления на дисковую подсистему не наблюдалось (Рисунок 3.5).

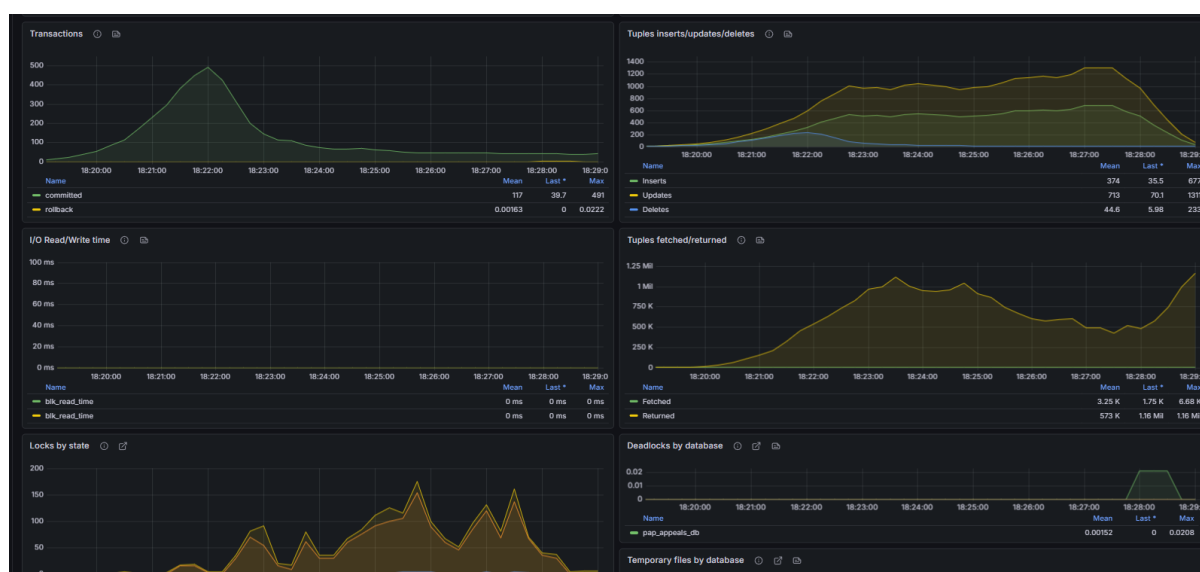


Рисунок 3.5 – Графики нагрузки на БД [разработано автором]

Дополнительно на графиках PostgreSQL фиксируется рост блокировочной активности: число блокировок в пиковые моменты достигало примерно 160–180. Также наблюдались единичные эпизоды deadlock, хотя они не носили массового характера. В целом это позволяет сделать вывод, что Transactional Outbox создаёт дополнительную транзакционную нагрузку на PostgreSQL, усиливает конкурентный доступ к данным и повышает вероятность накопления задержек при росте нагрузки.

3.5.2 Результаты по сценарию нагрузки для пакетной обработки на основе Spring Batch

По графику интенсивности HTTP-запросов видно, что в ходе теста нагрузка на сервис постепенно возрастала и выходила на устойчивый уровень

около 21–23 операций в секунду. К завершению теста интенсивность снижалась примерно до 6.56 операций в секунду, что соответствует этапу спада нагрузки. При этом на графике ошибок данные отсутствуют, то есть в рассматриваемом сценарии заметных всплесков HTTP 5xx не наблюдалось (Рисунок 3.6). График длительности HTTP-запросов показывает, что среднее время ответа оставалось низким и составляло около 5.19 мс, однако максимальная длительность отдельных запросов возрастала до 2.15 секунды. Такой всплеск связан с моментом запуска batch-задачи, когда приложение одновременно обслуживает входящие запросы и выполняет фоновую пакетную обработку накопленных данных.

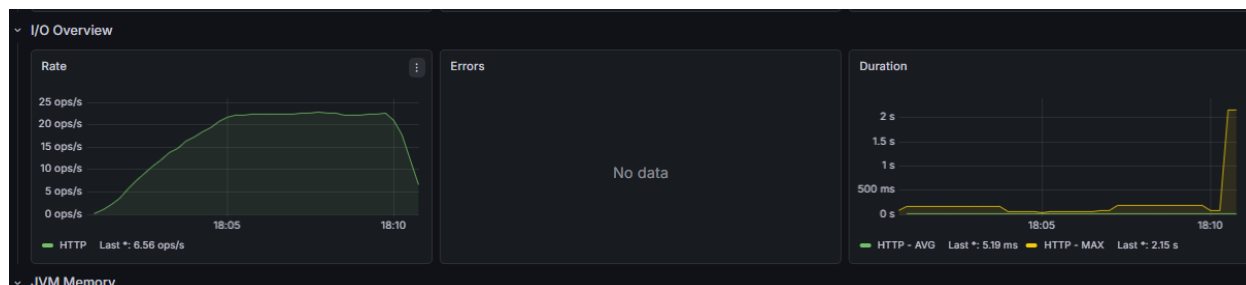


Рисунок 3.6 - Графики I/O Overview для пакетного подхода [разработано автором]

С точки зрения использования памяти JVM критических ограничений не наблюдалось. На графиках видно, что фактическое использование heap memory находилось на уровне около 57 MiB, а максимальное значение за тест достигало примерно 98.2 MiB. Общий объем используемой памяти JVM составлял около 222 MiB при максимуме примерно 253 MiB, что заметно ниже доступного предела. Показатели non-heap memory также оставались стабильными – около 165 MiB (Рисунок 3.7).

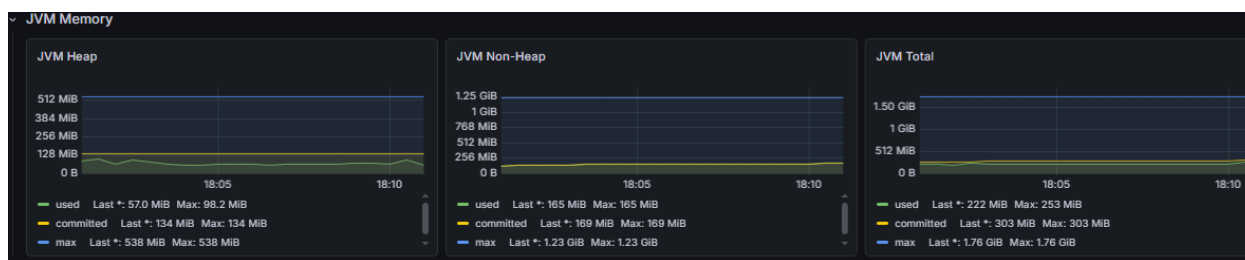


Рисунок 3.7 - Графики JVM Memory для пакетного подхода [разработано автором]

По графикам CPU и JVM Misc видно, что в основной части теста приложение работало стабильно и с невысокой загрузкой, однако в момент выполнения batch-задачи возникал кратковременный пик нагрузки. Загрузка процессора возрастала до 71.8%, показатель Load достигал примерно 1.9–2.0 при двух доступных CPU, а число потоков увеличивалось с 27–28 до 32–33. Это показывает, что пакетный подход создаёт не постоянную, а локальную дополнительную нагрузку на приложение именно в момент запуска фоновой обработки (Рисунок 3.8).

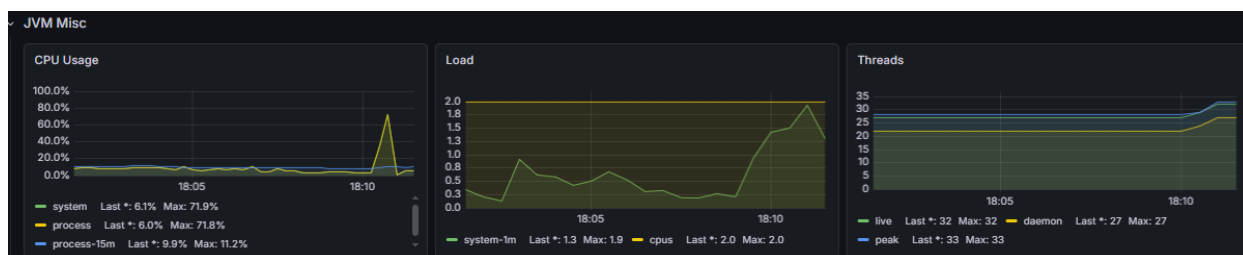


Рисунок 3.8 - Графики JVM Misc для пакетного подхода [разработано автором]

По графикам Kafka видно, что публикация сообщений происходит неравномерно, а сосредоточена в момент выполнения batch-задачи. На графике Message in per second интенсивность публикации в пике достигала примерно 54 сообщений в секунду, после чего быстро снижалась почти до нуля. На графике Message in per minute суммарная интенсивность возрастала до уровня около 683 сообщений в минуту (Рисунок 3.9). Такое поведение отражает саму природу пакетного подхода: данные сначала накапливаются, а затем отправляются в Kafka одной укрупнённой порцией.

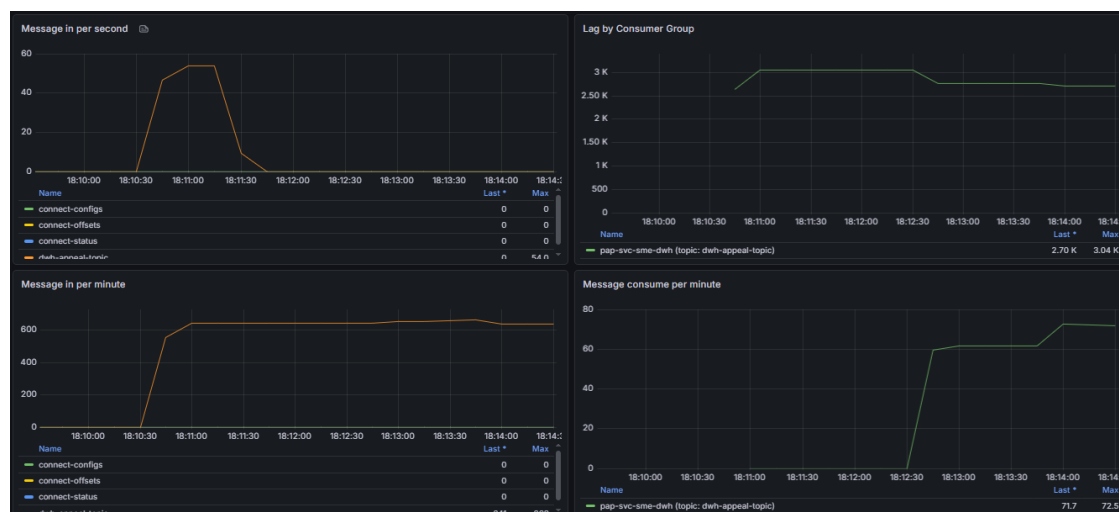


Рисунок 3.9 - Графики потока изменений в топике Kafka [разработано автором]

График Lag by Consumer Group показывает, что после публикации batch-пакета отставание потребителя возрастало примерно до 3.04 тыс. сообщений, а затем постепенно снижалось до 2.70 тыс. сообщений. При этом скорость потребления по графику Message consume per minute увеличивалась до 71–72 сообщений в минуту. Это говорит о том, что после пакетной отправки потребитель начинает разбирать накопленную очередь, однако не успевает полностью компенсировать объём данных, сформированный в момент batch-выгрузки.

Метрики PostgreSQL также показывают, что основная нагрузка на базу данных концентрируется именно в момент выполнения batch-задачи. Число commit-транзакций в пике достигало примерно 145, тогда как rollback оставался нулевым. Интенсивность операций insert, update и delete возрастала до 70–80 операций, а число возвращаемых строк на графике Tuples fetched/returned поднималось примерно до 109 тыс., что указывает на массовое чтение и обработку данных в рамках пакетной задачи. При этом I/O Read/Write time оставался практически на нулевом уровне, то есть выраженного давления на дисковую подсистему не наблюдалось (Рисунок 3.10).

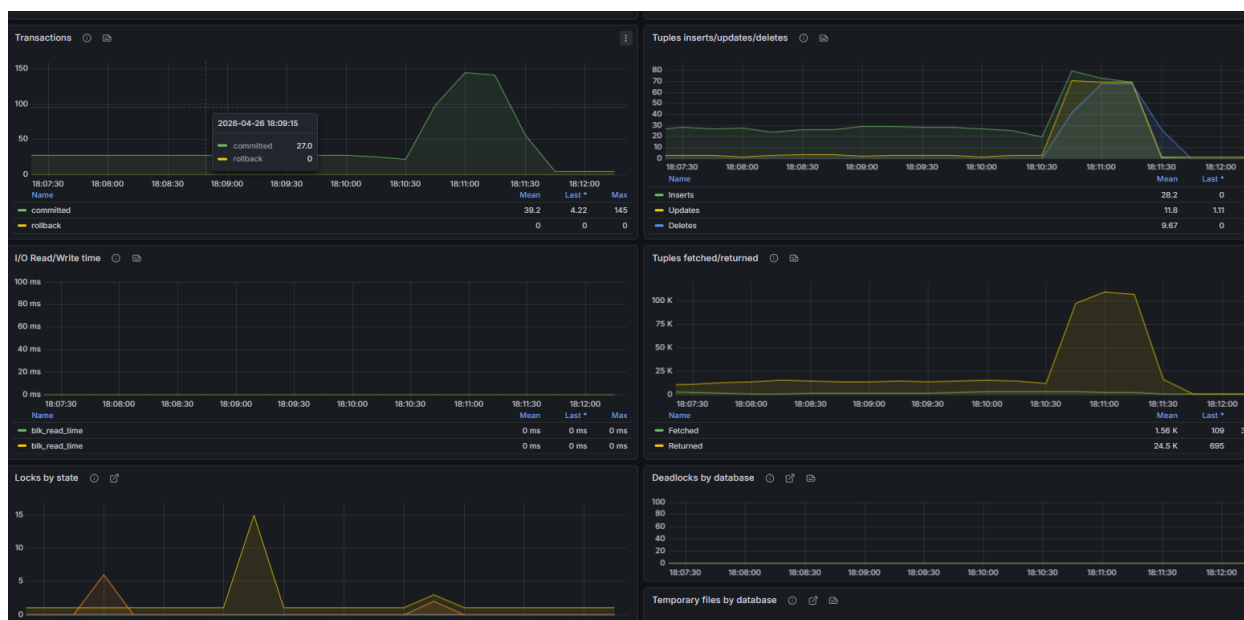


Рисунок 3.10 - Графики нагрузки на БД [разработано автором]

Дополнительно фиксировался кратковременный рост блокировочной активности. На графике Locks by state число блокировок в отдельные моменты

достигало примерно 15, что связано с конкурентным доступом к данным во время выполнения batch-задачи. При этом взаимоблокировки на графике Deadlocks by database не фиксировались. В целом это показывает, что пакетный подход создаёт нагрузку на PostgreSQL преимущественно импульсно: основная активность возникает в момент запуска фоновой задачи, когда происходит массовая выборка, обработка и отправка накопленных записей. Именно этот момент и объясняет кратковременное увеличение максимальной длительности HTTP-запросов.

3.5.3 Результаты по сценарию нагрузки для CDC репликации на основе Debezium

По графику интенсивности HTTP-запросов видно, что в ходе теста нагрузка на сервис постепенно возрастала и выходила на стабильный уровень около 22–23 операций в секунду. К завершению теста интенсивность сохранялась примерно на уровне 22.3 операций в секунду. При этом на графике ошибок данные отсутствуют, то есть в рассматриваемом сценарии заметных всплесков HTTP 5xx не наблюдалось (Рисунок 3.11). График длительности HTTP-запросов показывает, что работа Debezium практически не влияет на HTTP-запросы: среднее время ответа оставалось на уровне около 8.64 мс, а максимальная длительность к концу теста составляла около 53.9 мс. Даже пиковые значения укладывались в десятки миллисекунд, что говорит о стабильной обработке пользовательских запросов и отсутствии заметного влияния CDC-механизма на latency основного API.

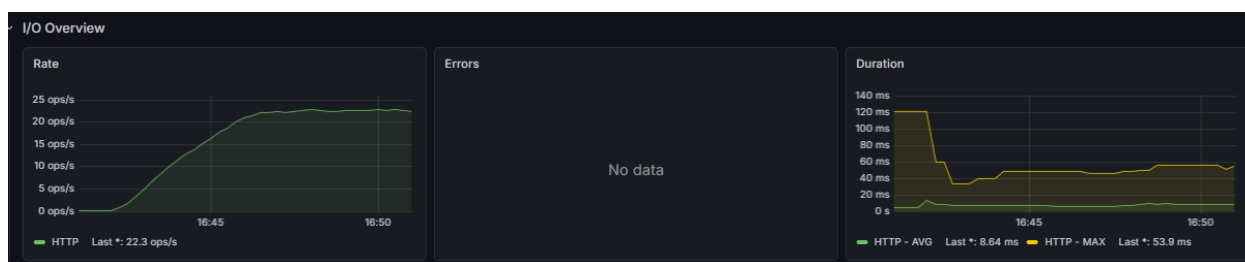


Рисунок 3.11 – Графики I/O Overview для подхода с Debezium [разработано автором]

С точки зрения использования памяти JVM критических ограничений также не наблюдалось. На графиках видно, что фактическое использование heap memory находилось на уровне около 84.6 MiB, а максимальное значение за тест достигало примерно 132 MiB. Общий объём используемой памяти JVM составлял около 249 MiB при максимуме примерно 296 MiB, что заметно ниже доступного предела. Показатели non-heap memory оставались стабильными – около 167 MiB (Рисунок 3.12).

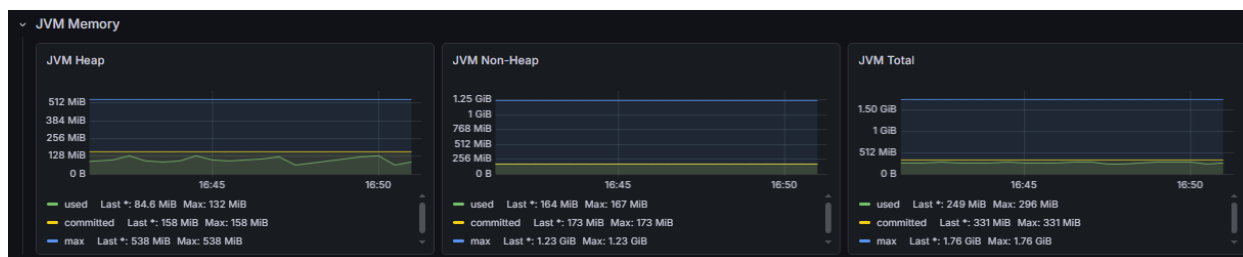


Рисунок 3.12 – Графики JVM Memory для подхода с Debezium [разработано автором]

По графикам CPU и JVM Misc видно, что приложение в этом сценарии работало спокойно и без выраженного роста нагрузки. Текущее значение CPU Usage составляло около 7.3%, а максимальное – примерно 7.9%. Показатель process-15m находился в пределах 7–12%, что также говорит о невысоком потреблении CPU самим приложением. При этом Load возрастал до 2.8 при двух доступных CPU, что уже отражает общую системную активность, связанную не столько с самим приложением, сколько с обработкой изменений и работой инфраструктурных компонентов. Число потоков оставалось практически стабильным: около 28 live threads при максимуме 31, количество daemon-потоков – около 23–26. Это показывает, что Debezium-подход не создаёт дополнительного давления на JVM и не приводит к заметному росту числа потоков в приложении (Рисунок 3.13).

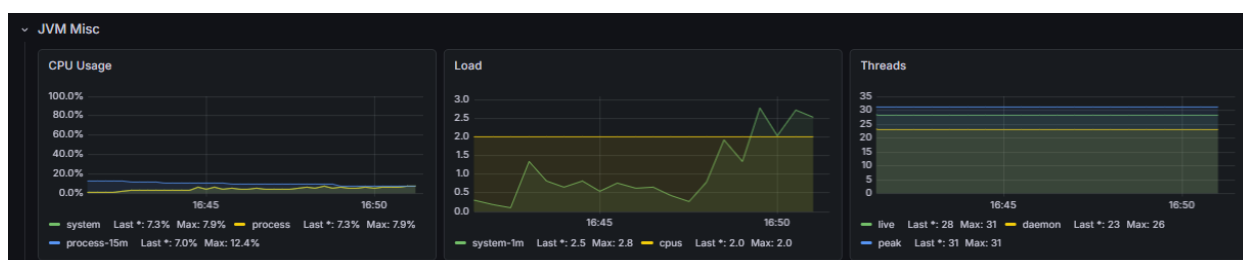


Рисунок 3.13 – Графики JVM Misc для подхода с Debezium [разработано автором]

По графикам Kafka видно, что поток сообщений формируется равномерно, без резких импульсных всплесков. На графике Message in per second интенсивность публикации по основным Debezium-топикам держалась примерно на уровне 6–7 сообщений в секунду, а к завершению теста снижалась примерно до 5 сообщений в секунду. На графике Message in per minute суммарная интенсивность публикации постепенно возрастала и достигала около 400 сообщений в минуту по основным топикам (Рисунок 3.14). В отличие от пакетного подхода, здесь изменения публикуются по мере их появления, поскольку Debezium считывает их непосредственно из журнала транзакций PostgreSQL.



Рисунок 3.14 – Графики потока изменений в топиках Kafka при использовании Debezium [разработано автором]

График Lag by Consumer Group показывает циклическое изменение отставания потребителей по нескольким sink-топикам. Для отдельных групп lag достигал примерно 396–405 сообщений, после чего быстро снижался до десятков сообщений. Это означает, что потребители в целом успевают регулярно разбирать накопленные изменения. На графике Message consume per minute видно, что скорость потребления достигала примерно 415–447 сообщений в минуту по разным sink-коннекторам. При этом потребление распределено сразу по нескольким топикам – appeal, appeal_change_history, initiator, responsible и

другим связанным таблицам. За счёт такого распределения снижается риск накопления большой очереди в одном топике.

Метрики PostgreSQL также показывают стабильную и предсказуемую нагрузку на базу данных источника. Число commit-транзакций держалось примерно на уровне 30–32 и к завершению снижалось до 25, тогда как rollback оставался равным нулю. Интенсивность insert в среднем составляла около 26.2 операций, максимальное значение достигало 28.8, а update находились на уровне 2–4 операций. Операции delete не фиксировались. Одновременно график Tuples fetched/returned показывает стабильное чтение данных: число возвращаемых строк находилось примерно в диапазоне 18–30 тыс., без резких скачков до сотен тысяч или миллионов строк. График I/O Read/Write time при этом оставался практически на нулевом уровне, то есть выраженного давления на дисковую подсистему не наблюдалось (Рисунок 3.15).



Рисунок 3.15 – Графики нагрузки на БД при использовании Debezium [разработано автором]

Дополнительно фиксировалась низкая блокировочная активность. На графике Locks by state наблюдались лишь кратковременные всплески, максимум около 18 блокировок, после чего значения быстро возвращались к базовому уровню. Взаимоблокировки на графике Deadlocks by database не фиксировались. В целом это показывает, что Debezium-подход почти не вмешивается в выполнение HTTP-запросов: основная логика приложения продолжает работать

стабильно, а передача изменений выносится на уровень CDC-чтения из журнала транзакций. Дополнительная нагрузка в этом случае проявляется прежде всего в работе инфраструктурного контура, а не в ухудшении времени ответа API.

3.6 Составление модели рекомендаций применения подходов

По результатам эксперимента видно, что рассмотренные подходы по-разному распределяют нагрузку между сервисом-источником, базой данных и инфраструктурным контуром. Transactional Outbox обеспечивает согласованную фиксацию бизнес-изменения и события на отправку, однако создаёт наибольшую дополнительную нагрузку на PostgreSQL и Kafka: в ходе теста наблюдались рост числа транзакций, высокая интенсивность insert/update-операций, увеличение блокировочной активности и накопление lag потребителя. Spring Batch оказывает меньшее влияние на сервис в обычный период работы, но формирует кратковременный пик нагрузки в момент запуска пакетной задачи: именно в этот момент возрастает длительность отдельных HTTP-запросов, увеличивается активность базы данных и происходит пакетная публикация сообщений в Kafka. Debezium CDC показал наименьшее влияние на HTTP-запросы: время ответа API оставалось стабильным, а передача изменений выполнялась за счёт инфраструктурного чтения WAL и последующей публикации событий через Kafka Connect. При этом основная сложность данного подхода переносится из прикладного кода в инфраструктурный контур.

На основе результатов нагрузочного тестирования и анализа подходов была сформирована модель выбора подхода к передаче изменений данных (Приложение Д.1).

3.7 Выводы по разделу

В рамках раздела были разработаны и экспериментально проверены три варианта передачи изменений данных: Transactional Outbox, Spring Batch и Debezium CDC. Для каждого решения было проведено нагрузочное тестирование, проанализировано влияние на бизнес-сервис, PostgreSQL и Kafka, после чего на основе полученных метрик сформирована модель рекомендаций по выбору подхода. Она показывает, что Debezium CDC целесообразен при необходимости минимизировать влияние на бизнес-сервис, Transactional Outbox – при необходимости прикладного контроля над формированием событий, а Spring Batch – при допустимой задержке доставки и возможности выполнять передачу данных в заранее выбранный период.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель: выполнено сравнение трёх подходов к передаче изменений данных на основе их реализации, нагрузочного тестирования и анализа влияния на компоненты распределённой информационной системы.

Для достижения цели были выполнены следующие задачи:

- проведён анализ предметной области и подходов к передаче данных;
- спроектированы и реализованы решения на основе Transactional Outbox, Spring Batch и Debezium CDC;
- разработана методика экспериментальной оценки реализованных решений;
- выполнено нагрузочное тестирование каждого подхода;
- проанализированы метрики производительности, полученные в ходе экспериментов;
- составлена модель рекомендаций по выбору подхода с учётом характеристик системы и ограничений инфраструктуры;

Полученные результаты показали, что каждый из рассмотренных подходов имеет собственную область применимости. Debezium CDC оказывает наименьшее влияние на пользовательские HTTP-запросы и обеспечивает более равномерную передачу изменений, но требует более сложной инфраструктурной настройки. Transactional Outbox позволяет гибко формировать доменные события на уровне приложения, однако создаёт дополнительную нагрузку на базу данных и Kafka. Spring Batch подходит для регламентной передачи данных при допустимой задержке, но формирует кратковременный пик нагрузки в момент выполнения пакетной задачи.

CONCLUSION

The final thesis achieved its stated objective: a comparison of three approaches to data change propagation was conducted based on their implementation, load testing, and impact analysis on distributed information system components.

To achieve this objective, the following tasks were completed:

- an analysis of the subject area and approaches to data transfer was conducted;
- solutions based on Transactional Outbox, Spring Batch, and Debezium CDC were designed and implemented;
- a methodology for experimentally evaluating the implemented solutions was developed;
- load testing of each approach was performed;
- performance metrics obtained during the experiments were analyzed;
- a recommendation model for selecting an approach was developed, taking into account system characteristics and infrastructure limitations.

The results showed that each of the approaches considered has its own area of applicability. Debezium CDC has the least impact on user HTTP requests and ensures more uniform change propagation, but requires a more complex infrastructure setup. Transactional Outbox allows for flexible generation of domain events at the application level, but creates additional load on the database and Kafka. Spring Batch is suitable for scheduled data transfer with acceptable latency, but creates a short-term peak load during batch task execution.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОНИЧКОВ

1. Richardson, C. Transactional outbox [Электронный ресурс] / С. Richardson. — URL: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обращения: 16.04.2026).
2. Richardson, C. Transaction log tailing [Электронный ресурс] / С. Richardson. — URL: <https://microservices.io/patterns/data/transaction-log-tailing.html> (дата обращения: 16.04.2026).
3. AWS. Transactional outbox pattern [Электронный ресурс]. — URL: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html> (дата обращения: 16.04.2026).
4. Hafiz, G. M. Transactional Outbox Pattern [Электронный ресурс] / G. M. Hafiz. — URL: <https://www.gmhafiz.com/blog/transactional-outbox-pattern/> (дата обращения: 16.04.2026).
5. Spring. Spring Batch [Электронный ресурс]. — URL: <https://spring.io/projects/spring-batch> (дата обращения: 16.04.2026).
6. Spring. Spring Batch Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-batch/reference/> (дата обращения: 16.04.2026).
7. Spring. Scaling and Parallel Processing — Spring Batch Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-batch/reference/scalability.html> (дата обращения: 16.04.2026).
8. IBM. What is Change Data Capture? [Электронный ресурс]. — URL: <https://www.ibm.com/think/topics/change-data-capture> (дата обращения: 16.04.2026).
9. Red Hat. What is change data capture? [Электронный ресурс]. — URL: <https://www.redhat.com/en/topics/integration/what-is-change-data-capture> (дата обращения: 16.04.2026).
10. Kleppmann, M. Designing Data-Intensive Applications [Электронный ресурс] / М. Kleppmann. — URL: <https://dataintensive.net/> (дата обращения: 16.04.2026).

11. Hohpe, G. Enterprise Integration Patterns [Электронный ресурс] / G. Hohpe, B. Woolf. — URL: <https://www.enterpriseintegrationpatterns.com/> (дата обращения: 16.04.2026).
12. Confluent. Message Delivery Guarantees for Apache Kafka [Электронный ресурс]. — URL: <https://docs.confluent.io/kafka/design/delivery-semantics.html> (дата обращения: 16.04.2026).
13. IBM. What is a data warehouse? [Электронный ресурс]. — URL: <https://www.ibm.com/think/topics/data-warehouse> (дата обращения: 16.04.2026).
14. Microsoft. Data warehousing in Microsoft Azure [Электронный ресурс]. — URL: <https://learn.microsoft.com/en-us/azure/architecture/data-guide/relational-data/data-warehousing> (дата обращения: 16.04.2026).
15. Brown, S. The C4 model for visualising software architecture [Электронный ресурс] / S. Brown. — URL: <https://c4model.com/> (дата обращения: 16.04.2026).
16. PlantUML. Sequence Diagram [Электронный ресурс]. — URL: <https://plantuml.com/sequence-diagram> (дата обращения: 16.04.2026).
17. Lucidchart. What is an Entity Relationship Diagram (ERD)? [Электронный ресурс]. — URL: <https://www.lucidchart.com/pages/er-diagrams> (дата обращения: 16.04.2026).
18. Krecan, L. ShedLock: Distributed lock for your scheduled tasks [Электронный ресурс] / L. Krecan. — URL: <https://github.com/lukas-krecan/shedlock> (дата обращения: 16.04.2026).
19. Baeldung. Guide to ShedLock with Spring [Электронный ресурс]. — URL: <https://www.baeldung.com/shedlock-spring> (дата обращения: 16.04.2026).
20. Debezium. Debezium PostgreSQL Source Connector for Confluent Platform [Электронный ресурс]. — URL: <https://docs.confluent.io/kafka-connectors/debezium-postgres-source/current/overview.html> (дата обращения: 16.04.2026).
21. Debezium. Debezium Connector for PostgreSQL [Электронный ресурс]. — URL:

<https://debezium.io/documentation/reference/stable/connectors/postgresql.html> (дата обращения: 16.04.2026).

22. Confluent. JDBC Sink Connector for Confluent Platform [Электронный ресурс]. — URL: <https://docs.confluent.io/kafka-connectors/jdbc/current/sink-connector/overview.html> (дата обращения: 16.04.2026).

23. Confluent. Oracle Database Sink Connector [Электронный ресурс]. — URL: <https://docs.confluent.io/cloud/current/connectors/cc-oracle-db-sink.html> (дата обращения: 16.04.2026).

24. Baeldung. JPA Entity Lifecycle Events [Электронный ресурс]. — URL: <https://www.baeldung.com/jpa-entity-lifecycle-events> (дата обращения: 16.04.2026).

25. Hibernate. Entity listeners and Callback methods [Электронный ресурс]. — URL: <https://docs.hibernate.org/stable/entitymanager/reference/en/html/listeners.html> (дата обращения: 16.04.2026).

26. Spring. TransactionSynchronizationManager [Электронный ресурс]. — URL: <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/support/TransactionSynchronizationManager.html> (дата обращения: 16.04.2026).

27. Spring. Scheduling Tasks [Электронный ресурс]. — URL: <https://spring.io/guides/gs/scheduling-tasks> (дата обращения: 16.04.2026).

28. Baeldung. The @Scheduled Annotation in Spring [Электронный ресурс]. — URL: <https://www.baeldung.com/spring-scheduled-tasks> (дата обращения: 16.04.2026).

29. Baeldung. Intro to Apache Kafka with Spring [Электронный ресурс]. — URL: <https://www.baeldung.com/spring-kafka> (дата обращения: 16.04.2026).

30. Docker. Docker Compose [Электронный ресурс]. — URL: <https://docs.docker.com/compose/> (дата обращения: 16.04.2026).

31. Docker. Compose file reference [Электронный ресурс]. — URL: <https://docs.docker.com/reference/compose-file/> (дата обращения: 16.04.2026).

32. Grafana k6. Grafana k6 Documentation [Электронный ресурс]. — URL: <https://grafana.com/docs/k6/latest/> (дата обращения: 16.04.2026).
33. Grafana k6. Ramping VUs Executor [Электронный ресурс]. — URL: <https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/ramping-vus/> (дата обращения: 16.04.2026).
34. Grafana k6. Thresholds [Электронный ресурс]. — URL: <https://grafana.com/docs/k6/latest/using-k6/thresholds/> (дата обращения: 16.04.2026).
35. Prometheus. Overview [Электронный ресурс]. — URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения: 16.04.2026).
36. Grafana. Dashboards [Электронный ресурс]. — URL: <https://grafana.com/docs/grafana/latest/visualizations/dashboards/> (дата обращения: 16.04.2026).
37. Spring. Metrics — Spring Boot Actuator [Электронный ресурс]. — URL: <https://docs.spring.io/spring-boot/reference/actuator/metrics.html> (дата обращения: 16.04.2026).

ПРИЛОЖЕНИЯ

Приложение А – Листинги кода транзакционного подхода

Приложение Б – Листинги кода пакетного подхода

Приложение В – Листинги кода CDC подхода

Приложение Г – Листинги кода настройки тестовой среды

Приложение Д – Модель рекомендаций

ПРИЛОЖЕНИЕ А

Листинги кода транзакционного подхода

Листинг А.1 – Слушатель изменений сущности [разработано автором]

```
@Component
@Slf4j
@RequiredArgsConstructor
public class AppealEntityListener {
    private static DwhService dwhService;

    @Autowired
    public AppealEntityListener(DwhService dwhService) {
        AppealEntityListener.dwhService = dwhService;
    }

    @PostPersist
    @PostUpdate
    public void postPersistOrUpdate(AppealEntity entity) {
        dwhService.registerSynchronization();
        dwhService.addToOutbox(String.valueOf(entity.getId()));
    }
}
```

Листинг А.2 – Сервис накопления изменений и подготовки данных [разработано автором]

```
@Service
@RequiredArgsConstructor
@Slf4j
public class DwhService {
    private static final String RESOURCE_KEY = "lks-outbox-appeals-ids";

    private final AppealService appealService;
    private final AppealMapper appealMapper;
    private final OutboxService outboxService;
    private final ChangeHistoryService changeHistoryService;

    private final TransactionSynchronizationAdapter
transactionSynchronizationAdapter = new TransactionSynchronizationAdapter() {
        @Override
        public void beforeCommit(boolean readOnly) {
            Set<String> ids = (Set<String>)
TransactionSynchronizationManager.getResource(RESOURCE_KEY);
            log.debug("Ids for dwh: {}", ids);
            List<AppealEntity> appeals = appealService.findAllById(ids);
            appeals.forEach(appeal ->
appeal.setHistories(changeHistoryService.findAllByAppealId(appeal.getId())));
            List<AppealDwhDTO> payload = appeals
}
```

Продолжение листинга А.2 [разработано автором]

```
        .stream()
        .map(appealMapper::toDwhDto)
        .toList();
    outboxService.processAppealDwh(payload);
}

};

public void addToOutbox(String appealId) {
    Set<String> ids = (Set<String>)
TransactionSynchronizationManager.getResource(RESOURCE_KEY);
    if (isNull(ids)) {
        ids = new LinkedHashSet<>();
        TransactionSynchronizationManager.bindResource(RESOURCE_KEY, ids);
    }
    ids.add(appealId);
}

public void registerSynchronization() {

TransactionSynchronizationManager.registerSynchronization(transactionSynchroniza
tionAdapter);
    }
}
```

Листинг А.3 – Формирование и сохранение записей в outbox [разработано автором]

```
@Transactional
public void processAppealDwh(List<AppealDwhDTO> appealDwhDTOS) {
    List<OutboxEntity> outboxEntities = appealDwhDTOS
        .stream()
        .map(appeal -> new OutboxEntity()
            .setMessageKey(appeal.getId().toString()))

    .setTopic(kafkaBindingsProperties.getDwhAppealProducerOut0().getDestination())
        .setPayload(writeValueAsString(new
OutboxDwhDTO().setPayload(new PayloadDwhDTO().setAppeal(appeal))))
        .toList();

    List<OutboxEntity> savedOutboxEntities = saveAll(outboxEntities);
    savedOutboxEntities.forEach(outboxEntity ->
    {
        OutboxDwhDTO outboxDwhDto =
readDwhValueFromJson(outboxEntity.getPayload());
        outboxDwhDto.setOutboxId(outboxEntity.getId());
        outboxDwhDto.setOutboxCreatedDate(outboxEntity.getCreatedDate());
        outboxEntity.setPayload(writeValueAsString(outboxDwhDto));
    });
    saveAll(savedOutboxEntities);
}
```

```
@Slf4j
@Service
@RequiredArgsConstructor
public class OutboxScannerJob extends AbstractJob {

    private final List<OutboxEventHandler> outboxEventHandlers;
    private final OutboxService outboxService;

    @Override
    @Scheduled(
        initialDelayString = "${outbox.initialDelayMs}",
        fixedRateString = "${outbox.fixedDelayMs}"
    )
    @SchedulerLock(
        name = "OutboxScannerJobLock",
        lockAtLeastFor = "${outbox.shedLock.lockAtLeastFor}",
        lockAtMostFor = "${outbox.shedLock.lockAtMostFor}"
    )
    protected void proceed() {
        log.trace("Outbox job start");
        boolean hasData = true;
        while (hasData) {
            List<OutboxEntity> outboxEntities = outboxService.findElements();
            if (!outboxEntities.isEmpty()) {
                outboxEntities.forEach(outboxEntity ->
                    outboxEventHandlers.stream()
                        .filter(outboxEventHandler ->
                            outboxEventHandler.isApplicable(outboxEntity))
                        .forEach(outboxEventHandler -> {
                            outboxEventHandler.handle(outboxEntity);
                            outboxService.delete(outboxEntity.getId());
                            log.trace("Outbox event was handled by: {}",
                                outboxEventHandler.getClass().getSimpleName());
                        }));
            } else {
                hasData = false;
            }
        }
        log.trace("Outbox job finished");
    }
}
```

```
@Slf4j
@RequiredArgsConstructor
public abstract class AbstractDwhRepository<T> {

    private final DwhUtilJdbcRepository dwhUtilRepository;
    private final JdbcTemplate jdbcTemplate;

    protected abstract String getTableName();

    protected abstract void addTableParameters(T payload, MapSqlParameterSource
sqlParameterSource);

    protected void save(DwhEntity<T> dwhEntity) {
        Assert.notNull(dwhEntity, "dwhEntity must not be null");
        log.debug("Try to save entity: {}", dwhEntity);

        MapSqlParameterSource mapSqlParameterSource =
buildDwhParameters(dwhEntity);
        addTableParameters(dwhEntity.getEntity(), mapSqlParameterSource);

        SimpleJdbcInsert simpleJdbcInsert = new SimpleJdbcInsert(jdbcTemplate)
            .withTableName(getTableName());
        int savedRows = simpleJdbcInsert.execute(mapSqlParameterSource);
        log.debug("{} row was saved successfully in table: {}", savedRows,
getTableName());
    }

    protected List<MapSqlParameterSource>
toSqlParameterSources(List<DwhEntity<T>> entities) {
        Assert.notEmpty(entities, "entities must not be empty");
        return entities.stream()
            .map(attribute -> {
                MapSqlParameterSource parameters =
buildDwhParameters(attribute);
                addTableParameters(attribute.getEntity(), parameters);
                return parameters;
            })
            .toList();
    }

    protected MapSqlParameterSource buildDwhParameters(DwhEntity<T> dwhEntity) {
        return new MapSqlParameterSource()
            .addValue("EVENT_GUID$", dwhEntity.getOutboxId())
            .addValue("SOURCE_CHANGE_TIME$",
toTimestamp(dwhEntity.getOutboxCreateDate()))
            .addValue("TARGET_CHANGE_TIME$", toTimestamp(Instant.now()))
            .addValue("TARGET_SEQUENCE_ID$",
dwhUtilRepository.getTargetSequenceId())
            .addValue("DML_FLG$", INSERT_DML_FLAG);
    }

    protected Timestamp toTimestamp(Instant instant) {
        if (instant == null) return null;
        return Timestamp.from(instant);
    }

    protected void saveAll(List<MapSqlParameterSource> parameterSources) {
        SimpleJdbcInsert simpleJdbcInsert = new SimpleJdbcInsert(jdbcTemplate)
```

Продолжение листинга А.5 [разработано автором]

```
        .withTableName(getTableName());
        int[] batchSavedRows =
simpleJdbcInsert.executeBatch(parameterSources.toArray(new
MapSqlParameterSource[0]));

        if (log.isDebugEnabled()) {
            long savedRows = Arrays.stream(batchSavedRows).count();
            log.debug("{} from {} rows was saved successfully in table: {}",
savedRows, parameterSources.size(), getTableName());
        }
    }
}
```

Листинг А.6 – Репозиторий загрузки обращений в Oracle [разработано автором]

```
@Slf4j
@Repository
@Transactional
public class AppealJdbcRepository extends AbstractDwhRepository<AppealDTO> {

    public AppealJdbcRepository(DwhUtilJdbcRepository dwhUtilRepository,
@Qualifier(DWH_JDBC_TEMPLATE) JdbcTemplate jdbcTemplate) {
        super(dwhUtilRepository, jdbcTemplate);
    }

    public void saveAppeal(DwhEntity<AppealDTO> appealData) {
        super.save(appealData);
    }

    @Override
    protected String getTableName() {
        return APPEAL_TABLE_NAME;
    }

    @Override
    protected void addTableParameters(AppealDTO appeal, MapSqlParameterSource
sqlParameterSource) {
        if (nonNull(appeal)) {
            sqlParameterSource
                .addValue("ID", appeal.getId())
                .addValue("NUMBER_TXT", appeal.getNumber())
                .addValue("CLIENT_HISTORY_ID", appeal.getClientId())
                .addValue("TYPE", appeal.getType())
                .addValue("TYPE_CODE", appeal.getTypeCode())
                .addValue("STATUS", appeal.getStatus())
                .addValue("STATUS_CODE", appeal.getStatusCode())
                .addValue("STAGE", appeal.getStage())
                .addValue("STAGE_CODE", appeal.getStageCode())
                .addValue("THEME", appeal.getTheme())
                .addValue("THEME_CODE", appeal.getThemeCode())
                .addValue("SUB_THEME", appeal.getSubTheme())
                .addValue("SUB_THEME_CODE", appeal.getSubThemeCode())
                .addValue("PRODUCT", appeal.getProduct())
                .addValue("PRODUCT_CODE", appeal.getProductCode())
                .addValue("PRODUCT_TYPE", appeal.getProductType())
                .addValue("PRODUCT_TYPE_CODE", appeal.getProductTypeCode())
                .addValue("CHANNEL", appeal.getChannel())
                .addValue("CHANNEL_CODE", appeal.getChannelCode())
                .addValue("PRIORITY", appeal.getPriority())
                .addValue("PRIORITY_CODE", appeal.getPriorityCode())
        }
    }
}
```

Продолжение листинга А.6 [разработано автором]

```
                .addValue("OPENED_DATE",
toTimestamp(appeal.getOpenedDate()))
                .addValue("CLOSED_DATE",
toTimestamp(appeal.getClosedDate()))
                .addValue("INITIATOR_ID",
Optional.of(appeal).map(AppealDTO::getInitiator).map(InitiatorDTO::getId).orElse
(null))
                .addValue("RESPONSIBLE_ID",
Optional.of(appeal).map(AppealDTO::getResponsible).map(ResponsibleDTO::getId).or
Else(null))
                .addValue("CLIENT_COMMENT", appeal.getClientComment())
                .addValue("CAUSE", appeal.getCause())
                .addValue("CALL_ID", appeal.getCallId());
        }
    }
}
```

ПРИЛОЖЕНИЕ Б

Листинги кода пакетного подхода

Листинг Б.1 – Конфигурация batch-задания [разработано автором]

```
@Configuration
@RequiredArgsConstructor
public class MigrateAppealJobConfig {

    private final JobRepository jobRepository;
    private final PlatformTransactionManager platformTransactionManager;
    private final BatchProperties batchProperties;

    private final AppealRepository appealRepository;

    @Bean
    public Job migrateAppealJob() {
        return new JobBuilder(MIGRATE_APPEAL_JOB_NAME)
            .repository(jobRepository)
            .incrementer(new RunIdIncrementer())
            .preventRestart()
            .start(migrateAppealStep())
            .build();
    }

    @Bean
    public Step migrateAppealStep() {
        return new StepBuilder(MIGRATE_APPEAL_STEP_NAME)
            .<AppealEntity,
AppealDwhDTO>chunk(batchProperties.getMigrateAppealsJob().getChunkSize())
            .reader(appealReader(WILL_BE_INJECTED))
            .repository(jobRepository)
            .transactionManager(platformTransactionManager)
            .build();
    }

    @Bean
    @StepScope
    public RepositoryItemReader<AppealEntity>
    appealReader(@Value("#{jobParameters}") Map<String, Object> jobParameters) {
        RepositoryItemReaderBuilder<AppealEntity> readerBuilder = new
        RepositoryItemReaderBuilder<AppealEntity>()
            .name("appealReader")
            .currentItemCount(0)
            .pageSize(batchProperties.getMigrateAppealsJob().getPageSize())
            .repository(appealRepository)
            .sorts(Map.of("lastUpdateDate", Sort.Direction.ASC));

        setReaderRepositoryMethod(jobParameters, readerBuilder);

        return readerBuilder.build();
    }

    private void setReaderRepositoryMethod(Map<String, Object> jobParameters,
    RepositoryItemReaderBuilder<AppealEntity> readerBuilder) {
        Instant now = Instant.now();
        MigrateAppealPeriod period =
        MigrateAppealPeriod.valueOf(jobParameters.get(PARAM_MIGRATION_PERIOD).toString()
        );
    }
}
```

Продолжение листинга Б.1 [разработано автором]

```
switch (period) {
    case DAILY -> {
        long lastUpdateDateIntervalForOpenedAppealInDays =
Long.parseLong(jobParameters.get(PARAM_LAST_UPDATE_DATE_INTERVAL_FOR_APPEAL_IN_D
AYS).toString());
        long lastUpdateDateIntervalForClosedAppealInDays =
Long.parseLong(jobParameters.get(PARAM_LAST_UPDATE_DATE_INTERVAL_FOR_CLOSED_APPE
AL_IN_DAYS).toString());

        Instant lastUpdateFrom =
now.minus(lastUpdateDateIntervalForOpenedAppealInDays, ChronoUnit.DAYS);
        Instant closedAppealLastUpdateFrom =
now.minus(lastUpdateDateIntervalForClosedAppealInDays, ChronoUnit.DAYS);
        readerBuilder
            .methodName("findAllInTwoIntervals")
            .arguments(lastUpdateFrom, now,
closedAppealLastUpdateFrom, now);
        log.info("Daily intervals will be used");
    }
    case MIGRATION -> {
        long lastUpdateDateIntervalForFullMigrationInDays =
Long.parseLong(jobParameters.get(PARAM_LAST_UPDATE_DATE_INTERVAL_FOR_FULL_MIGRAT
ION_IN_DAYS).toString());
        Instant lastUpdateFrom =
now.minus(lastUpdateDateIntervalForFullMigrationInDays, ChronoUnit.DAYS);
        readerBuilder
            .methodName("findAllInIntervals")
            .arguments(lastUpdateFrom, now);
        log.info("Full migration interval will be used");
    }
    default -> throw new IllegalStateException("Period is not supported:
" + period.name());
}
```

ПРИЛОЖЕНИЕ В

Листинги кода CDC подхода

Листинг В.1 – Конфигурация контейнера Kafka Connect в docker-compose [разработано автором]

```
kafka-connect:
  image: debezium/connect:2.5
  container_name: kafka-connect
  user: root
  depends_on:
    - kafka
    - zookeeper
  ports:
    - "8083:8083"
  environment:
    BOOTSTRAP_SERVERS: kafka:9092
    GROUP_ID: connect-cluster
    CONFIG_STORAGE_TOPIC: connect-configs
    OFFSET_STORAGE_TOPIC: connect-offsets
    STATUS_STORAGE_TOPIC: connect-status
    KEY_CONVERTER: org.apache.kafka.connect.json.JsonConverter
    VALUE_CONVERTER: org.apache.kafka.connect.json.JsonConverter
    CONNECT_REST_ADVERTISED_HOST_NAME: kafka-connect
    CONNECT_PLUGIN_PATH: /kafka/connect,/usr/share/java,/opt/connector/lib
    CLASSPATH: "/opt/connector/lib/*:/usr/share/java/kafka/*"
  volumes:
    - ./kafka-connect-plugins/oracle:/kafka/connect/oracle
    - ./kafka-connect-plugins/debezium-connector-jdbc:/kafka/connect/debezium-connector-jdbc
    - ./kafka-connect-logs:/var/log/kafka-connect
    - ./kafka-connect-plugins/oracle/ojdbc8-21.11.0.0.jar:/opt/connector/lib/ojdbc8.jar
  restart: unless-stopped
```

Листинг В.2 – Конфигурация source-коннектора Debezium для PostgreSQL

```
{
  "name": "pap-dialog-src",
  "config": {
    "connector.class":
"io.debezium.connector.postgresql.PostgresConnector",
    "tasks.max": 1,
    "plugin.name": "pgoutput",
    "publication.name": "dbz_publication_dialog",
    "database.hostname": "postgres",
    "database.port": 5432,
    "database.user": "debezium",
    "database.password": "dbz",
    "database.dbname": "pap_appeals_db",
    "database.server.name": "pg_server",
    "publication.autocreate.mode": "filtered"}
}
```

Продолжение листинга В.2 [разработано автором]

```
        "schema.include.list": "public",
        "table.include.list": "public.dialog",
        "transforms": "unwrap",
        "transforms.unwrap.type":
"io.debezium.transforms.ExtractNewRecordState",
        "transforms.unwrap.drop.tombstones": "true",
        "transforms.unwrap.add.fields":
"op,table,source.ts_ms,source.lsn,ts_ms",
        "heartbeat.interval.ms": 300000,
        "slot.name": "debezium_slot_dialog",
        "topic.prefix": "pg_server",
        "snapshot.mode": "initial",
        "message.key.columns": "public.dialog:id",
        "time.precision.mode": "connect",
        "topic.creation.enable": "true",
        "topic.creation.default.replication.factor": 1,
        "topic.creation.default.partitions": 1,
        "topic.creation.default.cleanup.policy": "compact",
        "topic.creation.default.compression.type": "lz4"
    }
}
```

Листинг В.3 – Конфигурация sink-коннектора JDBC для Oracle [разработано автором]

```
{
  "name": "shpp_dialog_sink",
  "config": {
    "connector.class": "io.debezium.connector.jdbc.JdbcSinkConnector",
    "tasks.max": "1",
    "dialect.name": "Oracle",
    "errors.log.enable": "true",
    "errors.log.include.messages": "true",
    "hibernate.dialect": "org.hibernate.dialect.OracleDialect",
    "hibernate.connection.driver_class": "oracle.jdbc.driver.OracleDriver",
    "connection.url": "jdbc:oracle:thin:@oracle:1521/XE",
    "connection.username": "DWH_STAGE_HIDDEN",
    "connection.password": "dwh_stage_password",
    "table.name.format": "DWH_STAGE_HIDDEN.SHPP_DIALOG",
    "topics": "pg_server.public.dialog",
    "transforms":
"unwrap,ReplaceField,RenameField,RenameKey,timestamp1,timestamp2,timestamp3,time
stamp4,timestamp5,Cast",

    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",

    "transforms.ReplaceField.type":
"org.apache.kafka.connect.transforms.ReplaceField$Value",
    "transforms.ReplaceField.renames":
"id:ID,appeal_id:APPEAL_ID,queue:QUEUE,external_theme_code:EXTERNAL_THEME_CODE,e
xternal_theme:EXTERNAL_THEME,external_id:EXTERNAL_ID",
```

Продолжение листинга В.3 [разработано автором]

```
"transforms.ReplaceField.blacklist": "__table",

    "transforms.RenameKey.type":
"org.apache.kafka.connect.transforms.ReplaceField$Key",
    "transforms.RenameKey.renames": "id:ID",

    "transforms.RenameField.type":
"org.apache.kafka.connect.transforms.ReplaceField$Value",
    "transforms.RenameField.renames":
"create_date:CREATE_DATE,update_date:UPDATE_DATE,__op:DML_FLG$,__source_ts_ms:SOURCE_CHANGE_TIME$,__source_lsn:TARGET_SEQUENCE_ID$,__ts_ms:TARGET_CHANGE_TIME$",

    "transforms.timestamp1.type":
"org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.timestamp1.field": "CREATE_DATE",
    "transforms.timestamp1.target.type": "Timestamp",

    "transforms.timestamp2.type":
"org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.timestamp2.field": "UPDATE_DATE",
    "transforms.timestamp2.target.type": "Timestamp",

    "transforms.timestamp3.type":
"org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.timestamp3.field": "SOURCE_CHANGE_TIME$",
    "transforms.timestamp3.target.type": "Timestamp",

    "transforms.timestamp4.type":
"org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.timestamp4.field": "__ts_ms",
    "transforms.timestamp4.target.type": "Timestamp",

    "transforms.timestamp5.type":
"org.apache.kafka.connect.transforms.TimestampConverter$Value",
    "transforms.timestamp5.field": "TARGET_CHANGE_TIME$",
    "transforms.timestamp5.target.type": "Timestamp",

    "transforms.Cast.type": "org.apache.kafka.connect.transforms.Cast$Value",
    "transforms.Cast.spec":
"ID:INT64,APPEAL_ID:INT64,QUEUE:INT64,EXTERNAL_THEME_CODE:string,EXTERNAL_THEME:string,EXTERNAL_ID:string,TARGET_SEQUENCE_ID$:INT64",

    "auto.create": "false",
    "auto.evolve": "false",
    "quote.identifiers": "true",
    "insert.mode": "upsert",
    "delete.enabled": "false",
    "schema.evolution": "none",
    "primary.key.mode": "record_key",
    "primary.key.fields": "ID",

    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "true"
}
}
```

ПРИЛОЖЕНИЕ Г

Листинги кода настройки тестовой среды

Листинг Г.1 – Основной фрагмент нагрузочного сценария [разработано автором]

```
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';

let errorRate = new Rate('errors');

let appealCreationTime = new Trend('appeal_creation_duration');
let appealSearchTime = new Trend('appeal_search_duration');
let appealGetTime = new Trend('appeal_get_duration');
let appealPatchTime = new Trend('appeal_patch_duration');
let appealBindingTime = new Trend('appeal_binding_duration');

export let options = {
  scenarios: {
    appeals_stable_load: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '1m', target: 10 },
        { duration: '1m', target: 20 },
        { duration: '2m', target: 35 },
        { duration: '5m', target: 35 },
        { duration: '1m', target: 0 },
      ],
      gracefulRampDown: '30s',
    },
  },
  thresholds: {
    http_req_failed: ['rate<0.04'],

    'http_req_duration{name:CreateAppeal}': ['p(95)<1500'],
    'http_req_duration{name:SearchByINN}': ['p(95)<800'],
    'http_req_duration{name:FilterSearch}': ['p(95)<1200'],
    'http_req_duration{name:GetByID}': ['p(95)<800'],
    'http_req_duration{name:PatchAppeal}': ['p(95)<1200'],
    'http_req_duration{name:BindingAppeal}': ['p(95)<1200'],

    appeal_creation_duration: ['p(95)<1500'],
    appeal_search_duration: ['p(95)<1200'],
    appeal_get_duration: ['p(95)<800'],
    appeal_patch_duration: ['p(95)<1200'],
    appeal_binding_duration: ['p(95)<1200'],

    errors: ['rate<0.04'],
  },
};

const BASE_URL = __ENV.APP_URL || 'http://localhost:8080/api/v1/appeals';
let createdAppealIds = [];

function markResult(success) {
  errorRate.add(!success);
}

export default function () {
  const random = Math.random();
```

Продолжение листинга Г.1 [разработано автором]

```
// 30% – создание обращения
if (random < 0.30) {
    const appeal = generateAppeal();

    const start = Date.now();

    const res = http.post(`${BASE_URL}`, JSON.stringify(appeal), {
        headers: { 'Content-Type': 'application/json' },
        tags: { name: 'CreateAppeal' },
        timeout: '20s',
    });

    appealCreationTime.add(Date.now() - start);

    const success = check(res, {
        'create status 200': r => r.status === 200,
    });

    if (success) {
        const body = JSON.parse(res.body);
        createdAppealIds.push(body.id);
    }

    markResult(success);
}

// 35% – поиск обращения по ИНН
else if (random < 0.65) {
    const start = Date.now();

    const res = http.get(`${BASE_URL}?inn=${generateInn()}`, {
        tags: { name: 'SearchByINN' },
        timeout: '10s',
    });

    appealSearchTime.add(Date.now() - start);

    const success = check(res, {
        'search status 200': r => r.status === 200,
    });

    markResult(success);
}

// 25% – поиск обращений с фильтрами
else if (random < 0.90) {
    const queryString = buildQueryString(generateSearchCriteria());
    const start = Date.now();

    const res = http.get(`${BASE_URL}/filter${queryString}`, {
        tags: { name: 'FilterSearch' },
        timeout: '20s',
    });

    appealSearchTime.add(Date.now() - start);

    const success = check(res, {
        'filter status 200': r => r.status === 200,
    });

    markResult(success);
}
```

```
// 10% – операции с ранее созданными обращениями
else {
  if (createdAppealIds.length === 0) {
    return;
  }

  const      appealId      =      createdAppealIds[Math.floor(Math.random()
createdAppealIds.length)];
  const operation = Math.random();

  if (operation < 0.60) {
    const start = Date.now();

    const res = http.get(`${BASE_URL}/${appealId}`, {
      tags: { name: 'GetByID' },
      timeout: '10s',
    });

    appealGetTime.add(Date.now() - start);

    markResult(check(res, {
      'get status 200': r => r.status === 200,
    }));
  } else if (operation < 0.85) {
    const start = Date.now();

    const      res      =      http.patch(`${BASE_URL}/${appealId}`,
JSON.stringify(generatePatchRequest(appealId)), {
      headers: { 'Content-Type': 'application/json' },
      tags: { name: 'PatchAppeal' },
      timeout: '20s',
    });

    appealPatchTime.add(Date.now() - start);

    markResult(check(res, {
      'patch status 200': r => r.status === 200,
    }));
  } else {
    const start = Date.now();

    const      res      =      http.patch(`${BASE_URL}/${appealId}/inn`,
JSON.stringify(generateBindingRequest()), {
      headers: { 'Content-Type': 'application/json' },
      tags: { name: 'BindingAppeal' },
      timeout: '20s',
    });

    appealBindingTime.add(Date.now() - start);

    markResult(check(res, {
      'binding status 200': r => r.status === 200,
    }));
  }
}

sleep(Math.random() * 1.5 + 0.8);
}
```

Листинг Г.2 – Конфигурация контейнеров тестовой среды [разработано автором]

```
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  volumes:
    - ./pap-appeals-sme-develop/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.console.libraries=/usr/share/prometheus/console_libraries'
    - '--web.console.templates=/usr/share/prometheus/consoles'
    - '--web.enable-lifecycle'
  ports:
    - "9090:9090"
  networks:
    - pap-network
  restart: unless-stopped

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  volumes:
    - grafana_data:/var/lib/grafana
    - ./grafana/provisioning/datasources:/etc/grafana/provisioning/datasources
    - ./grafana/provisioning/dashboards:/etc/grafana/provisioning/dashboards
    - ./grafana/dashboards:/var/lib/grafana/dashboards
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_INSTALL_PLUGINS=grafana-piechart-panel
    - GF_FEATURE_TOGGLES_ENABLE=tempoApmTable
    - GF_AUTH_ANONYMOUS_ENABLED=false
    - GF_LOG_LEVEL=debug
  ports:
    - "3000:3000"
  networks:
    - pap-network
  depends_on:
    - prometheus
  restart: unless-stopped

node-exporter:
  image: prom/node-exporter:latest
  container_name: node-exporter
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /:/rootfs:ro
  command:
    - '--path.procfs=/host/proc'
    - '--path.sysfs=/host/sys'
    - '--path.rootfs=/rootfs'
    - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($|/)'
  ports:
    - "9100:9100"
  networks:
    - pap-network
```

```
restart: unless-stopped

kafka-exporter:
  image: danielqsj/kafka-exporter:latest
  container_name: kafka-exporter
  command:
    - '--kafka.server=kafka:9092'
    - '--web.listen-address=:9308'
    - '--log.level=debug'
  ports:
    - "9308:9308"
  networks:
    - pap-network
  depends_on:
    - kafka
  restart: unless-stopped
  healthcheck:
    test: ["CMD", "wget", "--spider", "-q", "http://localhost:9308/metrics"]
    interval: 10s
    timeout: 5s
    retries: 5

postgres-exporter:
  image: prometheuscommunity/postgres-exporter:latest
  container_name: postgres-exporter
  environment:
    DATA_SOURCE_NAME:
      "postgresql://postgres:postgres@postgres:5432/pap_appeals_db?sslmode=disable"
  ports:
    - "9187:9187"
  networks:
    - pap-network
  depends_on:
    postgres:
      condition: service_healthy
  restart: unless-stopped
```

ПРИЛОЖЕНИЕ Д

Модель рекомендаций

Таблица Д.1 – Итоговая модель рекомендаций

Условие / Ограничение системы	Транзакционный подход	Пакетный подход	CDC подход
Высокая нагрузка на бизнес-сервис	Не рекомендуется: дополнительные записи в outbox-таблицу увеличивают транзакционную активность и вероятность блокировок, что может приводить к росту времени ответа пользовательских запросов	Ограниченно применим (при условии запуска вне пиковых часов): нагрузка на бизнес-сервис отсутствует, но требуется планирование окон выполнения	Рекомендуется: чтение изменений из WAL выполняется вне основного потока транзакций и оказывает минимальное влияние на прикладной сервис
Высокая нагрузка на PostgreSQL	Не рекомендуется: дополнительные записи в outbox-таблицу увеличивают количество транзакций, операций записи и блокировок	Ограниченно применим: нагрузка концентрируется в момент выполнения batch-задачи и при корректном выборе окна запуска остаётся допустимой	Рекомендуется: логическая репликация создаёт минимальную дополнительную нагрузку на БД
Ограниченные вычислительные ресурсы приложения	Не рекомендуется: формирование и публикация событий создают постоянную дополнительную нагрузку на CPU, которая может конкурировать с выполнением бизнес-логики	Ограниченно применим: batch-задача вызывает значительный пик CPU, однако он контролируем и может быть вынесен за пределы пиковой нагрузки	Рекомендуется: практически не увеличивает нагрузку на CPU бизнес-сервиса
Высокие требования к времени доставки изменений	Не рекомендуется: при росте нагрузки увеличивается задержка доставки вследствие накопления очереди сообщений	Не рекомендуется: доставка зависит от расписания запуска batch-задачи и не обеспечивает низкую задержку	Рекомендуется: обеспечивает наиболее стабильную доставку изменений с минимальной задержкой

Продолжение таблицы Д.1

Ограниченная пропускная способность Kafka-потребителей	Не рекомендуется: при высокой интенсивности изменений возможно быстрое накопление consumer lag вследствие формирования большого потока сообщений	Ограниченно применим: lag возникает преимущественно в момент пакетной выгрузки, а объём передаваемых данных контролируется настройками batch-задачи	Рекомендуется: более равномерный поток сообщений снижает вероятность накопления критического consumer lag
Требование гибкого формата сообщения	Рекомендуется: payload формируется в прикладном коде и может содержать любые бизнес-данные без дополнительных преобразований	Рекомендуется: позволяет выполнять фильтрацию, агрегацию и обогащение данных перед отправкой	Не рекомендуется: сообщения отражают структуру изменений в таблицах и требуют дополнительного слоя обработки для формирования сложных бизнес-событий
Чувствительность к количеству соединений с БД	Ограниченно применим: фоновый публикатор создаёт дополнительную нагрузку на пул соединений, особенно при конкурентной обработке	Ограниченно применим: batch-задача создаёт пик соединений и запросов в момент выполнения	Рекомендуется: использует отдельное репликационное соединение и практически не влияет на пул соединений приложения
Требование минимизации изменений в существующем коде бизнес-сервиса	Не рекомендуется: требует интеграции механизма формирования и сохранения outbox-событий в прикладной код	Ограниченно применим: требует реализации batch-логики в приложении или выделенном сервисе обработки	Рекомендуется: бизнес-сервис остаётся неизменным, а изменения считываются из журнала транзакций PostgreSQL
Сложность сопровождения инфраструктуры (DevOps, мониторинг, отладка)	Средняя: требуется контроль дублей, настройка повторных отправок, мониторинг outbox-таблицы и очередей сообщений	Средняя: требуется сопровождение расписания запуска, мониторинг выполнения batch-задач и обработка сбоев	Высокая: требует развёртывания и сопровождения Kafka Connect, настройки коннекторов, мониторинга логической репликации и позиции чтения WAL